

TINDAHAN POS

Technical System Documentation

Prepared in support of the BIR accreditation / registration process

Dells Software

Document version 1.0 · Status: DRAFT · 31 August 2026

COMPLIANCE NOTICE

Tindahan POS is currently in ALPHA and is **NOT BIR-accredited**. This document describes the current implementation based on inspected source code and deployed configuration. Documentation of a technical control does not by itself constitute BIR accreditation, certification, approval, or legal compliance. Final compliance status and applicable requirements must be determined through the appropriate BIR process and applicable regulations.

1. Document control

Field	Value
Document title	Tindahan POS Technical Documentation
Software	Tindahan POS
Software provider	Dells Software
Document version	1.0
Software version	apps/tindahan-pos 0.0.0 ; apps/tindahan-pos-mobile 1.0.0 – see §37
Date	2026-08-31
Status	DRAFT
Repository revision	dev @ 56befe0
Prepared by	[TO BE VERIFIED]
Reviewed by	[TO BE VERIFIED]
Approved by	[TO BE VERIFIED]

Revision history

Version	Date	Author	Change
1.0	2026-08-31	[TO BE VERIFIED]	Initial technical documentation compiled from source inspection

How to read the status markers

Marker	Meaning
IMPLEMENTED	Verified present in source or database
PARTIALLY IMPLEMENTED	Present but incomplete for the stated purpose
NOT IMPLEMENTED	Verified absent
FUTURE / PLANNED	Intended, not built
FOR BIR VALIDATION	Requires BIR/RDO or tax-professional confirmation
[TO BE VERIFIED]	Could not be verified from the repository

Every technical claim below was checked against the source tree, migrations, or the live staging database. Where verification was not possible it is marked, not guessed.

2. Project information

Field	Value	Status
Project name	Tindahan POS	IMPLEMENTED
Software provider	Dells Software	IMPLEMENTED
Product description	Point-of-sale and inventory system for Philippine sari-sari stores and small retailers, including customer credit ("utang") management	IMPLEMENTED
System type	Multi-tenant SaaS; web application + companion mobile application, shared managed backend	IMPLEMENTED
Current status	ALPHA / BIR ACCREDITATION PREPARATION	IMPLEMENTED
Supported platforms	Modern desktop and mobile web browsers; iOS and Android via Expo (see §26)	IMPLEMENTED
Repository	Monorepo, npm workspaces (<code>apps/*</code> , mobile excluded from the workspace)	IMPLEMENTED
Technical contact	[TO BE VERIFIED]	—
Business contact	[TO BE VERIFIED]	—
Company registration / TIN	[TO BE VERIFIED]	FOR BIR VALIDATION

Version numbering caveat. `apps/tindahan-pos` and `apps/super-admin` both carry `version: 0.0.0` in `package.json` — a scaffold default, not a release version. There is no release-tagging scheme in the repository. A meaningful software version must be established before submission; see §36 (Missing documentation).

3. Executive summary

Tindahan POS is a point-of-sale and inventory system aimed at Philippine sari-sari stores and small retailers. It records sales, deducts stock, tracks customer credit balances, and produces sales and inventory reporting. Each store is an isolated tenant.

Implemented today

- Web POS with cash, QR, and credit ("utang") payment types
- Product, category, and inventory management including receiving, stock adjustment, counts, warehouses, transfers, and purchase orders
- Customer credit with per-customer limits, payments, and aging
- Staff, roles, and granular permissions; cashier PIN sessions; paired counter devices
- Reporting including a **Z-Reading** with reconciliation
- Immutable audit log covering 12 distinct verified action types
- A companion mobile application (owner/admin oriented)
- Alpha/Test print guardrails on every print surface

Not implemented

- **X-Reading** — verified absent
- Payment gateway / electronic payment capture — no provider integrated
- Accounting, payroll, multi-branch consolidation, AI features

Alpha and BIR status

The application prints an **ORDER SLIP** marked * **TEST MODE / TRAINING ONLY** * and * **NOT AN OFFICIAL BIR INVOICE/RECEIPT** *. It does not issue, and must not be represented as issuing, official BIR invoices or receipts. See §15 and §18.

4. System scope

Included and implemented

Authentication · registration · staff and role management · store management · products · categories · inventory (receiving, adjustments, counts, warehouses, transfers, purchase orders, unit conversions) · POS checkout · sales · payments (cash/QR/credit) · customers · credit/utang · cashier PIN sessions · paired devices · reports incl. Z-Reading · dashboard · printing and reprinting · CSV/JSON export · audit log · settings · demo store · subscription/trial state · web application · mobile application.

Excluded

Area	Status
Accounting / bookkeeping module	FUTURE / PLANNED
Payroll	NOT IMPLEMENTED
Multi-branch consolidation	PARTIALLY IMPLEMENTED — <code>core.branch.manage</code> permission and warehouse/transfer tables exist; consolidated branch reporting does not
AI assistant	FUTURE / PLANNED
Payment gateway integration	NOT IMPLEMENTED
E-invoicing / BIR electronic transmission	NOT IMPLEMENTED — FOR BIR VALIDATION

5. System architecture

5.1 Frontend (web)

Aspect	Implementation
Framework	React 19.2.4
Language	TypeScript ~6.0.2
Build	Vite ^8.1.1
Routing	react-router-dom ^7.18.2
Styling	Tailwind CSS ^4.3.3 + project CSS tokens (<code>tpl-*</code>)
State	React hooks and context; no external state library
Data access	@supabase/supabase-js ^2.110.7
Testing	Vitest ^4.1.10, Playwright ^1.61.1

5.2 Mobile

Aspect	Implementation
Framework	React Native 0.81.5 on Expo ^54.0.36
Language	TypeScript ~5.9.3
Navigation	Local component state , not a router library — verified: no navigation package; <code>App.tsx</code> branches on state
Styling	NativeWind ^4.2.6 (Tailwind ^3.4.19)
Local storage	@react-native-async-storage/async-storage , expo-secure-store
Device capabilities	Camera (barcode), image picker/manipulator, document picker, file system, sharing, crypto
Printing	NOT IMPLEMENTED — see §18.4
Testing	Jest ^29.7.0 (<code>jest-expo</code>) + React Native Testing Library

5.3 Backend

Tindakan POS has **no separately deployed application server**. The backend is Supabase (managed PostgreSQL + PostgREST + GoTrue auth + Storage + Edge Functions).

Aspect	Implementation
API architecture	PostgREST auto-generated REST over PostgreSQL, plus PostgreSQL functions invoked as RPC
Business logic	In the database — PL/pgSQL functions (e.g. <code>checkout_sale</code> , <code>void_sale</code> , <code>adjust_product_stock</code>)
Authentication	Supabase Auth (GoTrue), JWT bearer tokens
Authorization	PostgreSQL Row Level Security + <code>has_permission()</code> checks
Custom server code	6 Deno Edge Functions (§28)

Architectural consequence worth stating for BIR purposes: because transaction logic lives in database functions and authorization lives in RLS, the controls are enforced below the client. A modified or hostile client

cannot bypass them. This is evidenced in §12 and §15.

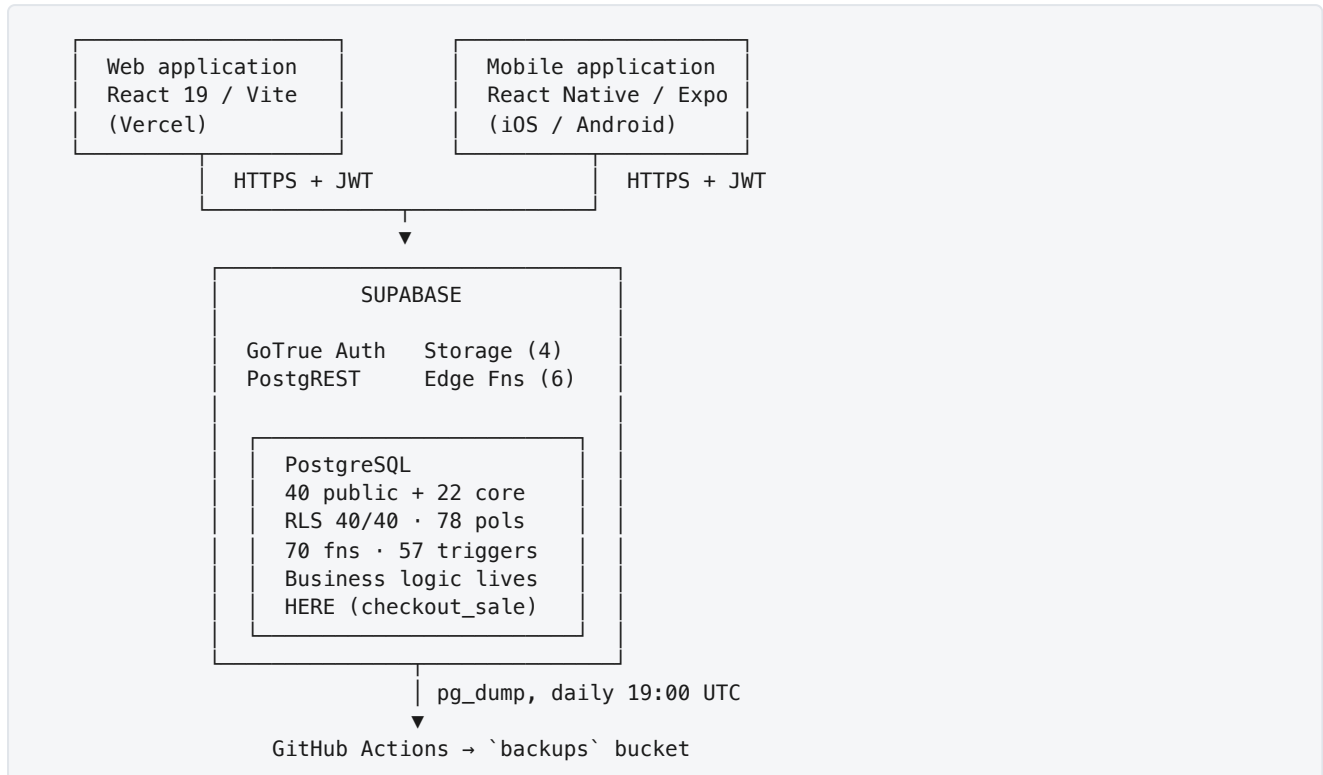
5.4 Database

Metric	Verified value
Engine	PostgreSQL (Supabase managed)
Tables — <code>public</code> schema	40
Tables — <code>core</code> schema	22
Tables with RLS enabled	40 of 40 (100%)
RLS policies	78
Functions (<code>public</code>)	70
Triggers (non-internal)	57
Foreign key constraints	84
Migrations	120

5.5 Infrastructure

Aspect	Implementation	Status
Web hosting	Vercel (<code>vercel.json</code> present)	IMPLEMENTED
Database / auth / storage	Supabase (managed)	IMPLEMENTED
Storage buckets	<code>avatars</code> (public), <code>product-images</code> (public), <code>store-photos</code> (public), <code>backups</code> (private)	IMPLEMENTED
TLS	Provided by Vercel and Supabase	IMPLEMENTED
Security headers	CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy in <code>vercel.json</code>	IMPLEMENTED
CI	GitHub Actions — <code>tindakan-pos-ci.yml</code> , <code>platform-ci.yml</code>	IMPLEMENTED
Scheduled backup	GitHub Actions — <code>backup-production.yml</code> , daily <code>0 19 * * *</code> UTC	IMPLEMENTED
Monitoring / uptime / alerting	[TO BE VERIFIED] — no configuration found in the repository	—
CDN / DNS	Vercel defaults; custom domain configuration [TO BE VERIFIED]	—

5.6 Architecture diagram



6. Technology stack

Layer	Technology	Version	Purpose
Web	React	19.2.4	Web application
Web build	Vite	^8.1.1	Bundling
Web routing	react-router-dom	^7.18.2	Client routing
Mobile	React Native	0.81.5	Mobile application
Mobile platform	Expo	^54.0.36	Native runtime and tooling
Language	TypeScript	~6.0.2 (web) / ~5.9.3 (mobile)	Type safety
Backend / DB	Supabase (PostgreSQL)	managed	Data, auth, API, storage
Client SDK	@supabase/supabase-js	^2.110.7 / ^2.111.0	API access
Auth	Supabase Auth (GoTrue)	managed	Identity
Hosting	Vercel	managed	Web deployment
Storage	Supabase Storage	managed	Images, backups
Testing (web)	Vitest	^4.1.10	Unit / integration
Testing (e2e)	Playwright	^1.61.1	End-to-end
Testing (mobile)	Jest + jest-expo	^29.7.0	Unit / component
Styling	Tailwind CSS	^4.3.3 / ^3.4.19	Styling

Versions are taken from the applications' `package.json` files at the documented revision. Exact resolved versions are pinned in the repository lockfile.

7. Application modules

7.1 Authentication — IMPLEMENTED

Aspect	Detail
Provider	Supabase Auth (GoTrue)
Password storage	Handled by GoTrue; the application never stores or hashes passwords itself
Sessions	JWT access token + refresh token, persisted by the Supabase client
Registration	Self-service owner signup; a <code>handle_new_user()</code> trigger provisions the staff row and store
Password reset	Implemented (email link → set new password)
Logout	Implemented, and emits <code>staff_logged_out</code> to the audit log
Cashier PIN	Separate PIN session on shared devices (<code>cashier_sessions</code> , <code>set_own_pin</code>)
Paired device	Device identity with no staff row (§11)
Google OAuth	PARTIALLY IMPLEMENTED — client-side support exists; the provider is not enabled in the Supabase project, so the flow fails. [TO BE VERIFIED] before release
MFA	Implemented in the Super Admin application only, not the POS

7.2 User management, staff and roles — IMPLEMENTED

Two layered role systems, both verified in the database:

1. **Structural role** — `staff.role`, values `admin` or `cashier` only.
2. **Granular RBAC** — `roles` (`OWNER`, `SUPERVISOR`, `CASHIER`), `permissions` (20 codes), `role_permissions` (35 grants), `staff_roles` assignment.

Staff creation for cashiers/supervisors goes through the `create-cashier` Edge Function; the role picker cannot create another owner.

7.3 Products, categories, inventory — IMPLEMENTED

Tables: `products`, `categories`, `warehouses`, `warehouse_stock`, `warehouse_transfers`, `receiving_entries`, `receiving_lines`, `inventory_counts`, `inventory_count_lines`, `inventory_beginning_balances`, `product_unit_conversions`, `purchase_orders`, `purchase_order_lines`, `suppliers`, `supplier_categories`, `stock_discrepancies`.

Product fields include name, barcode, category, price, stock, low-stock threshold, pack quantity/price, cost, image. Stock adjustment is performed by `adjust_product_stock()` — a database function, so concurrent adjustments are serialized by the database rather than by the client.

7.4 POS / checkout — IMPLEMENTED

See §8 for the full transaction description and sequence diagram.

7.5 Customers and credit (utang) — IMPLEMENTED

See §9.

7.6 Reporting — IMPLEMENTED

See §20 and §21.

7.7 Settings — IMPLEMENTED

Profile, store details, receipts, fees & limits, alerts, backup, devices, plan, audit log.

8. Sales transaction logic

8.1 Where it runs

Checkout is the PostgreSQL function `checkout_sale()`, called as an RPC. It is `SECURITY DEFINER`. Its parameters, verified from the database:

```
p_items jsonb, p_services jsonb, p_customer_id uuid, p_payment_type text,  
p_reference_no text, p_override_pin text, p_cashier_token text,  
p_client_request_id uuid, p_occurred_at timestamptz,  
p_is_offline_replay boolean, p_discount_type text,  
p_discount_value numeric, p_override_token text
```

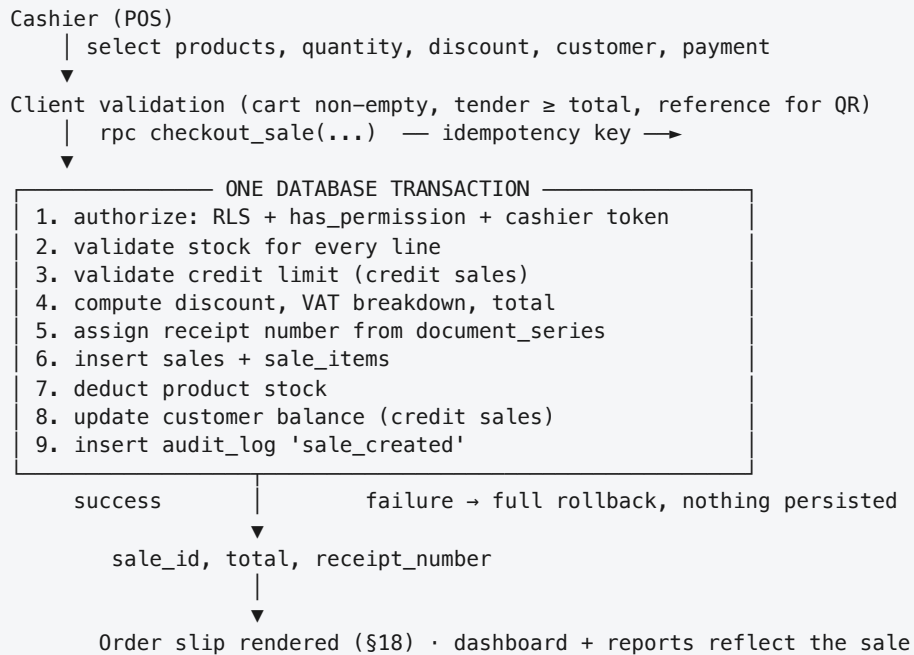
8.2 Recorded per sale

`sales` : id, store, timestamp, receipt number, cashier name, payment type, reference number, customer, total, discount type/value/amount, VAT status and breakdown (vatable, VAT amount, exempt, zero-rated), status (completed / voided), void metadata. `sale_items` : product, name, quantity, unit price, item type, fee, line total.

8.3 Atomicity, rollback, duplicate prevention

Control	Mechanism	Status
Atomicity	The whole sale — insert, stock deduction, customer balance, receipt number, audit row — executes inside one PL/pgSQL function, therefore one transaction. Any failure rolls the entire sale back.	IMPLEMENTED
Duplicate prevention (server)	<code>p_client_request_id</code> idempotency key	IMPLEMENTED
Duplicate prevention (client)	The checkout button is disabled while in flight and the handler re-checks on entry, so a double-tap cannot fire twice	IMPLEMENTED
Stock validation	Enforced inside the function, not the client	IMPLEMENTED
Credit limit	Enforced inside the function; raises <code>CREDIT_LIMIT_EXCEEDED</code> ; a null limit means unlimited	IMPLEMENTED
Owner PIN override	<code>p_override_pin</code> / <code>p_override_token</code> , with lockout	IMPLEMENTED

8.4 Transaction sequence



9. Customer / utang management — IMPLEMENTED

Aspect	Implementation
Customer record	<code>customers</code> : name, phone, credit limit (nullable = unlimited), balance
Balance authority	Server-maintained. Balance is never client-settable
Increase	A credit sale inside <code>checkout_sale()</code>
Decrease	<code>credit_payments</code> — payment rows with a recorded resulting balance
Limit enforcement	Inside <code>checkout_sale()</code> ; <code>CREDIT_LIMIT_EXCEEDED</code>
Owner override	<code>credit_overrides</code> , <code>credit_override_tokens</code> , PIN with lockout
Aging	Implemented (debt-age reporting)
Protection	RLS store-scoping + server-side computation

10. Payment management

Method	Status	Notes
Cash	IMPLEMENTED	Tendered and change recorded
QR (GCash/Maya)	IMPLEMENTED	Manual entry of a reference number. No gateway integration — the app does not verify the payment with any provider
Credit / utang	IMPLEMENTED	Balance updated atomically
Card / terminal	NOT IMPLEMENTED	
Online payment gateway	NOT IMPLEMENTED	
Refund	IMPLEMENTED	<code>refunds</code> , <code>refund_items</code> , append-only; <code>pos.sale.refund</code>
Void	IMPLEMENTED	<code>void_sale()</code> ; reverses stock and utang; audit row with previous value

FOR BIR VALIDATION: QR payments record an operator-typed reference only. Whether that satisfies BIR evidentiary expectations for non-cash tender is not determined here.

11. Cashier, device and shift management

Aspect	Status	Detail
Cashier accounts	IMPLEMENTED	Created via <code>create-cashier</code> Edge Function
Cashier PIN	IMPLEMENTED	<code>set_own_pin</code> ; PIN hash stored, never the PIN
PIN session	IMPLEMENTED	<code>cashier_sessions</code> ; audit events <code>cashier_session_started</code> / <code>_ended</code>
Device pairing	IMPLEMENTED	<code>devices</code> , <code>device_pairing_codes</code> , <code>pair-device</code> Edge Function
Device identity	IMPLEMENTED	A paired device has an auth user but no <code>staff</code> row ; it resolves its store through <code>devices</code>
Device revocation	IMPLEMENTED	<code>unpair-device</code> sets <code>unpaired_at</code> and deletes the auth user . Verified: an already-issued token loses all data access immediately, because <code>auth_store_id()</code> re-evaluates <code>unpaired_at</code> is <code>null</code> on every query
Shift open/close, starting cash, cash variance	PARTIALLY IMPLEMENTED	Cashier *sessions* and drawer-variance surfaces exist. A formal shift lifecycle with declared opening float and blind close was not verified as a complete implementation — [TO BE VERIFIED]

12. RBAC — verified matrix

Derived directly from the `roles`, `permissions`, `role_permissions` tables, not from documentation.

Roles: OWNER, SUPERVISOR, CASHIER. **Permissions:** 20 codes. **Grants:** 35 rows. OWNER = 20 permissions · SUPERVISOR = 15 · CASHIER = **0 rows** (a cashier holds no granular permissions).

Permission	OWNER	SUPERVISOR	CASHIER
core.audit.view	Y	Y	–
core.branch.manage	Y	–	–
core.organization.manage	Y	–	–
core.staff.assign_role	Y	–	–
core.staff.create	Y	–	–
core.staff.view	Y	Y	–
customer.manage	Y	Y	–
inventory.product.manage	Y	Y	–
inventory.purchase_order.manage	Y	Y	–
inventory.stock.adjust	Y	Y	–
inventory.stock.count	Y	Y	–
inventory.stock.receive	Y	Y	–
inventory.supplier.manage	Y	Y	–
inventory.transfer.manage	Y	Y	–
inventory.warehouse.manage	Y	Y	–
pos.report.view	Y	Y	–
pos.sale.refund	Y	Y	–
pos.sale.void	Y	Y	–
settings.store.manage	Y	Y	–
staff.manage	Y	–	–

SUPERVISOR lacks exactly: `core.branch.manage`, `core.organization.manage`, `core.staff.assign_role`, `core.staff.create`, `staff.manage`.

Notes:

- An admin (`staff.role = 'admin'`) passes `has_permission()` unconditionally and needs no `staff_roles` row.
- A CASHIER performs POS functions through role-based RLS policies, not through granular permissions — holding zero permission grants is by design, not a gap.
- **settings.store.manage is granted to SUPERVISOR by design.** A supervisor can therefore change store settings including service-fee pricing. This is an intentional grant recorded in `role_permissions`, and is now covered by the audit trail (§17).

13. Database design

40 tables in `public`, 22 in `core`, **all RLS-enabled**, 84 foreign keys, 78 policies, 57 triggers.

13.1 Table inventory (`public`)

Identity and tenancy: `stores`, `staff`, `roles`, `permissions`, `role_permissions`, `staff_roles`, `devices`, `device_pairing_codes`, `cashier_sessions`, `feature_flags`

Catalogue and stock: `products`, `categories`, `suppliers`, `supplier_categories`, `warehouses`, `warehouse_stock`, `warehouse_transfers`, `product_unit_conversions`, `inventory_beginning_balances`, `inventory_counts`, `inventory_count_lines`, `receiving_entries`, `receiving_lines`, `purchase_orders`, `purchase_order_lines`, `stock_discrepancies`

Transactions: `sales`, `sale_items`, `refunds`, `refund_items`, `document_series`

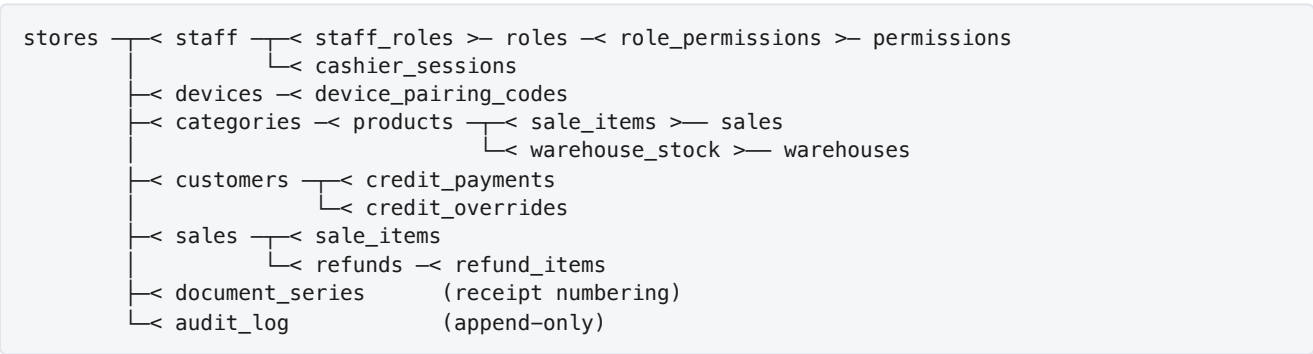
Credit: `customers`, `credit_payments`, `credit_overrides`, `credit_override_tokens`

Governance: `audit_log`

Demo/marketing (isolated): `demo_products`, `demo_sales`, `demo_customers`, `demo_requests`

The `core` schema (22 tables) carries the multi-tenant organization, subscription, plan, entitlement and platform-audit model.

13.2 Entity relationships (core transactional path)



13.3 Integrity controls

Control	Evidence
Referential integrity	84 foreign keys
Tenant scoping	<code>store_id</code> on tenant tables + RLS on all 40
Server-computed money	Totals, VAT, balances computed in <code>checkout_sale()</code>
Audit immutability	<code>AUDIT_LOG_IMMUTABLE</code> trigger — verified: UPDATE and DELETE both refused (42501)
Receipt uniqueness	<code>document_series.next_number</code> allocated server-side
Price-edit guard	<code>guard_cashier_product_update()</code> raises <code>PRICE_EDIT_NOT_ALLOWED</code>

14. Data flow

Login

```
Credentials → Supabase Auth → JWT → auth_store_id() resolves tenant
→ staff row (role) → staff_roles → permissions → UI gating
→ audit_log 'staff_logged_in'
```

Sale

See the sequence diagram in §8.4.

Inventory

```
Receiving / adjustment / count → adjust_product_stock() (DB function)
→ products.stock → low-stock threshold evaluation → restock surfaces
```

Credit

```
Credit sale → checkout_sale() → customers.balance ↑
Payment      → credit_payments (records resulting balance) → balance ↓
→ aging and payment history
```

15. Security architecture

15.1 Authentication

Control	Status	Evidence
Managed identity provider	IMPLEMENTED	Supabase Auth (GoTrue)
Password hashing	IMPLEMENTED (delegated)	GoTrue; app never handles hashes
JWT sessions + refresh	IMPLEMENTED	Supabase client
Password reset	IMPLEMENTED	Email link flow
Cashier PIN hashing	IMPLEMENTED	<code>staff.pin_hash</code> ; PIN never stored plaintext
Session revocation on unpair	IMPLEMENTED	Auth user deleted; access dies immediately
MFA on the POS application	NOT IMPLEMENTED	Present in Super Admin only
Rate limiting on auth	[TO BE VERIFIED]	Supabase platform defaults; not configured in-repo

15.2 Authorization

Control	Status	Evidence
Database-enforced RLS	IMPLEMENTED	78 policies; 40/40 tables
Permission checks server-side	IMPLEMENTED	<code>has_permission()</code> in policies and functions
Client-side gating	IMPLEMENTED	Defence in depth only, never the boundary
Verified by test	IMPLEMENTED	A CASHIER receives zero rows from <code>sales</code> via the API — an empty result set from the database, not a filtered screen

15.3 Tenant isolation — verified by live testing

Tested against staging with **real user JWTs** over the REST API (not with elevated database credentials, which bypass RLS):

Test	Result
Own-scope reads return only the caller's store	PASS
Explicit cross-tenant read by <code>store_id</code>	0 rows , all roles, all tables
Cross-tenant INSERT	HTTP 403
Cross-tenant UPDATE	0 rows
Anonymous access with anon key, no user JWT	Refused (42501) on every table

The `staff` table holds every tenant's staff emails; no role saw beyond its own store.

15.4 Application security

Control	Status	Evidence
SQL injection	IMPLEMENTED	Parameterized PostgREST/RPC; no string-built SQL in app code
XSS	IMPLEMENTED	React escaping; the report print builder uses <code>createElement / textContent</code> only, never <code>innerHTML</code>
CSP, HSTS, X-Frame-Options, Referrer-Policy, Permissions-Policy	IMPLEMENTED	<code>vercel.json</code>
Secrets management	IMPLEMENTED	Environment variables; no credentials committed (repository history verified clean)
CSRF	NOT APPLICABLE	Bearer tokens, not cookie-session auth
Rate limiting (application)	NOT IMPLEMENTED	

15.5 Known security-relevant finding

A SUSPENDED organization can still complete sales — HIGH, open. `core.org_writes_allowed()` returns false for a suspended organization and its own comment states the intent as "suspended = read-only". It is not read-only: the 27 policies enforcing `writes_allowed` sit on inventory and purchasing tables only. `products`, `sales` and `sale_items` have none, and `checkout_sale()` is `SECURITY DEFINER` and never consults billing state.

Verified by reproduction: with the organization suspended, a product update returned HTTP 204 and `checkout_sale` completed a sale and issued a receipt number.

This is a **billing/entitlement** defect, not a tenant-isolation defect — isolation held throughout. It is disclosed here rather than omitted. Status: **open, pending a product decision.**

16. Data privacy

Personal data held

Category	Data	Location
Staff	Name, email, phone, address, avatar, PIN hash	staff , avatars bucket
Customer	Name, phone, credit limit, balance, transaction history	customers , sales , credit_payments
Transaction	Items, amounts, cashier, timestamps	sales , sale_items
Authentication	Email, hashed credentials	Supabase Auth

Technical controls (IMPLEMENTED)

Tenant isolation by RLS on every table; TLS in transit; encryption at rest by the platform; account deletion implemented end-to-end (delete-account and approve-deletion-request Edge Functions, with foreign-key coverage across all referencing tables verified); data export available to the operator (CSV/JSON).

Legal / regulatory requirements — NOT ASSESSED HERE

Philippine **Data Privacy Act (RA 10173)** obligations — lawful basis, privacy notice, consent records, NPC registration, data-processing agreements, breach notification procedures, and retention schedules — are **legal requirements, not technical controls**. They are outside the scope of this document and are **[TO BE VERIFIED]** with counsel or the NPC. The presence of a technical control does not establish legal compliance.

17. Audit trail — IMPLEMENTED

`audit_log` columns: `id`, `store_id`, `actor_id`, `action`, `entity_type`, `entity_id`, `previous_value`, `new_value`, `reason`, `created_at`.

Verified action types (counts from the live staging database)

Action	Rows	Category
<code>staff_logged_in</code>	115	Authentication
<code>cashier_session_started</code>	42	Session
<code>staff_logged_out</code>	26	Authentication
<code>sale_created</code>	20	Transaction
<code>cashier_session_ended</code>	13	Session
<code>sale_voided</code>	7	Transaction
<code>store_config_changed</code>	5	Configuration
<code>staff_role_changed</code>	5	Authorization
<code>receipt_reprinted</code>	3	Printing
<code>store_settings_updated</code>	2	Configuration
<code>sale_refunded</code>	1	Transaction
<code>price_changed</code>	1	Catalogue

Reprints are audited (`receipt_reprinted`) — relevant to BIR expectations around duplicate documents.

Tamper resistance

An immutability trigger raises `AUDIT_LOG_IMMUTABLE`. Verified live: UPDATE and DELETE against audit rows are both refused with `42501`. Read access is limited to admins and holders of `pos.report.view`.

Gaps

Gap	Status
No IP address or user-agent captured	NOT IMPLEMENTED — the table has no such columns
Inventory adjustments not represented among observed actions	[TO BE VERIFIED] — may be unexercised in staging rather than unimplemented
Retention policy	NOT IMPLEMENTED — no defined retention period; FOR BIR VALIDATION

18. Invoice / printing system

18.1 Current Alpha output

The application does **not** produce an official invoice or receipt. It produces an **ORDER SLIP** carrying mandatory, non-removable markings.

```
*** TEST MODE / TRAINING ONLY ***

      QA TEST STORE
    Store address not set yet

      ORDER SLIP

-----
000017 · 01 Aug 9:14 AM
Cashier: Maricel
-----
Pancit Canton x3          54.00
Skyflakes x2             18.00
Globe load 100           100.00
Service fee               5.00
-----
TOTAL                     177.00
Cash                     200.00
Change                   23.00
-----
      Salamat po! Balik kayo ulit.

*** NOT AN OFFICIAL BIR INVOICE/RECEIPT ***
```

18.2 Print surfaces (all guarded)

#	Surface	Output method
1	Receipt.tsx via ReceiptModal	window.print() — browser, A4, thermal, print-to-PDF
2	printReport.ts	window.open document — report modals, supplier sheets
3	ReceiptPreviewPanel	Settings → Receipts preview
4	Mobile ReceiptPreview	Preview only

There is **no separate PDF generator** — "Save as PDF" is a destination of the same browser print dialog, so a PDF is the marked document rendered to file.

18.3 Enforcement

Mode is centralized (`VITE_APP_MODE` / `EXP0_PUBLIC_APP_MODE`). Unset, unknown, or `BIR` all resolve to `ALPHA` ; only `PRODUCTION` disables the automatic disclaimer, and `PRODUCTION` **does not mean accredited**.

Each surface calls `printGuardrails()` itself rather than receiving it as a prop, so no call site can construct an unmarked document. The header renders before the VOIDED and REPRINT markers; the footer renders after the operator's own footer message. In ALPHA the document title replaces the store's `invoice_type` , TIN and permit number are withheld even for a store flagged `birRegistered` , and VAT sections are hidden. The TIN & permit setting is **locked**, not merely ignored.

Reprints retain both disclaimers plus `* REPRINT *` , and are audited.

18.4 Mobile printing — NOT IMPLEMENTED

Mobile has no print pathway: no browser print, no thermal driver. It has a guarded *preview* and delivery toggles ("Print on the thermal printer", "Print automatically every sale") which persist to local storage and **drive no implementation**. A future driver must render through the guarded component or it becomes an unmarked surface.

18.5 Future production mode — FUTURE / PLANNED / FOR BIR VALIDATION

Required BIR document content, layout, wording, and any electronic transmission are **not implemented and not validated**. No PTU, accreditation number, MIN, SIN, ACN, permit number, or serial range exists anywhere in the system — none was invented.

19. Transaction numbering

Aspect	Implementation	Status
Source	<code>document_series (store_id , series_key , prefix , next_number)</code>	IMPLEMENTED
Scope	Per store, per series key (<code>default</code> observed)	IMPLEMENTED
Format	Zero-padded 6-digit (e.g. <code>000017</code>), optional prefix (empty in observed data)	IMPLEMENTED
Allocation	Server-side within <code>checkout_sale()</code> 's transaction	IMPLEMENTED
Concurrency	Allocation inside the sale transaction; two sales cannot share a number	IMPLEMENTED
Offline sales	Number assigned on sync; the receipt shows a pending placeholder until then	IMPLEMENTED
Voids	Number retained; the sale is marked <code>voided</code> , never deleted	IMPLEMENTED
Reprints	Reuse the original number and are audited	IMPLEMENTED
Failed transactions	Rolled back — a failed sale consumes no number	IMPLEMENTED
Reset behaviour	[TO BE VERIFIED] — no periodic reset was identified	
BIR conformity of format, series and reset rules	[REQUIRES BIR VALIDATION]	

20. Reporting system – IMPLEMENTED

Report	Source	Access	Export
Daily sales / sales by transaction	sales , sale_items	pos.report.view	Print, CSV
Payment breakdown	sales	pos.report.view	Print
Cashier breakdown	sales , cashier_sessions	pos.report.view	Print
VAT summary	sales VAT columns	pos.report.view	Print
Void summary	sales (voided)	pos.report.view	Print
Refund summary	refunds	pos.report.view	Print
Z-Reading	aggregate (§21)	pos.report.view	Print
Best sellers	sale_items	admin	Print
Low stock / restocking	products	admin	Print
Utang / customer balances / aging	customers , credit_payments	admin	Print
Data export (sales, products, everything)	live tables	admin	CSV / JSON

Reports are filterable by date range, cashier, and device.

21. X-Reading / Z-Reading

Capability	Status
Z-Reading	IMPLEMENTED
X-Reading	NOT IMPLEMENTED — verified absent
Daily closing	PARTIALLY IMPLEMENTED — Z-Reading exists; a formal end-of-day close procedure was not verified
Cashier summary	IMPLEMENTED
Payment summary	IMPLEMENTED
Void summary	IMPLEMENTED
Discount summary	IMPLEMENTED (Z-Reading total discounts)
Tax summary	IMPLEMENTED (VAT summary + Z-Reading)
Audit journal	IMPLEMENTED (<code>audit_log</code> , §17)
Electronic journal	PARTIALLY IMPLEMENTED (§22)

Z-Reading fields verified in source: business date, total sales, transaction count, **beginning receipt number**, **ending receipt number**, total discounts, VATable sales, VAT amount, voided count, voided total, payment-type breakdown, and a **reconciliation check** with match/mismatch states.

FOR BIR VALIDATION: whether this Z-Reading satisfies the required accumulating totals (grand total, reset counter, Z-counter) has **not** been determined. Those specific fields were not identified in the implementation — see §36.

22. Electronic journal — PARTIALLY IMPLEMENTED

A complete, queryable transaction history exists: `sales` + `sale_items` are retained permanently, voids retain their rows and numbers, refunds are append-only, and `audit_log` is immutable and covers 12 verified action types.

What a BIR-conforming **Electronic Journal** additionally implies — a defined record format, sequential journal export, tamper-evident sealing (e.g. hash chaining), and a stated retention period — is **NOT IMPLEMENTED**.

Proposed implementation (FUTURE / PLANNED)

1. A journal export producing every transaction for a date range in a fixed, documented record format.
2. Hash chaining across journal entries so any alteration is detectable.
3. A stated retention period aligned to BIR requirements, with enforcement and restore testing.
4. Read-only journal access for examination, distinct from operational reporting.

23. Backup and disaster recovery

Aspect	Verified	Status
Mechanism	pg_dump via GitHub Actions (backup-production.yml)	IMPLEMENTED
Frequency	Daily, cron 0 19 * * * (UTC)	IMPLEMENTED
Destination	Private Supabase Storage bucket backups	IMPLEMENTED
Client access	None — RLS denies all client roles; the bucket is not public	IMPLEMENTED
Platform backups	Supabase-managed backups per plan	[TO BE VERIFIED]
Retention period	[TO BE VERIFIED] — not defined in the workflow	
Restore procedure	NOT DOCUMENTED	
Restore testing	NO EVIDENCE — no record of a tested restore	
RPO / RTO	NOT DEFINED	
Redundancy / failover	[TO BE VERIFIED]	

A backup that has never been restored is an untested control. This is called out in §36 as a gap.

24. Error handling

Class	Behaviour	Status
Validation	Field-level messages; submission blocked	IMPLEMENTED
Authentication	Explicit sign-in failures	IMPLEMENTED
Authorization	Server refusal (RLS/ 42501 /403), surfaced as a permission message	IMPLEMENTED
Database business rules	Named exceptions (CREDIT_LIMIT_EXCEEDED , PRICE_EDIT_NOT_ALLOWED , AUDIT_LOG_IMMUTABLE) mapped to operator-facing copy	IMPLEMENTED
Network failure at checkout	Sale queued offline (§25) rather than lost	IMPLEMENTED
Duplicate submission	Idempotency key + disabled-while-pending control	IMPLEMENTED
Partial failure	Impossible for a sale — one transaction, full rollback	IMPLEMENTED
Printing errors	Surfaced with the action re-enabled for retry	IMPLEMENTED
Centralized error monitoring	NOT IMPLEMENTED — no reporting service configured	

25. Offline / network behaviour — IMPLEMENTED (web)

Aspect	Implementation
Offline queue	<code>src/lib/offlineQueue</code> (web)
Behaviour	Checkout continues when the connection drops; the sale is stored on the device
Statuses	pending, syncing, synced, needs cashier re-auth, failed
Sync	Automatic on reconnection; replay flagged via <code>p_is_offline_replay</code>
Duplicate prevention on replay	Idempotency key
Receipt number	Assigned on sync; placeholder shown until then
Visibility	Queue surfaced in Settings → Backup with a per-sale status list
Mobile offline queue	NOT IMPLEMENTED

26. Mobile application

Platform: iOS and Android via Expo. Navigation is local component state — there is no router library.

Screen	Purpose	Role	Data source	Actions
Splash	Session restore	all	Supabase Auth	—
Login	Sign in	all	Auth	Sign in, Google (provider not enabled)
Create account	Owner signup	public	Auth	Register
Pair device	Register a counter device	public	pair-device	Pair with code
Cashier PIN	PIN entry on a paired device	cashier	cashier_sessions	Start session
Onboarding	First-run wizard	admin	stores , products	Complete setup
Demo store	Read-only sample store	any	demo tables	Browse
Owner home	Dashboard	admin	sales , products , customers	Drill-down
Today's sales	Daily sales	admin	sales	View, open insights
Insights	Sales analysis	admin	sales , sale_items	View
POS	Register	admin, cashier, device	products , checkout_sale	Sell
Restock	Low stock and restocking	admin	products	Adjust
Utang	Customer credit	admin	customers , credit_payments	Record payment
Pricing	Plans	admin	plan_prices()	Start trial
Trial expired	Post-trial gate	admin	billing state	Choose plan
Setup register	Device management	admin	devices	Pair / unpair
Settings — menu	Settings hub	admin	—	Navigate
Settings — profile	Name, phone, avatar, password, PIN	any staff	staff	Edit
Settings — store	Store details, BIR fields	admin	stores	Edit
Settings — receipts	Order-slip settings and preview	admin	stores , local	Edit
Settings — fees & limits	Service-fee brackets	admin	stores.fee_config	Edit
Settings — alerts	Alert thresholds	admin	local storage	Edit
Settings — backup	Counts, refresh, CSV/JSON export	admin	live tables	Export via share sheet

Mobile limitations: no printing (§18.4); no offline queue; several settings areas persist to local storage only, mirroring the web application's own mock areas; Settings is admin-only (a cashier is routed straight to the register).

27. Web application — screen inventory

Routes verified from `src/App.tsx`.

Route	Purpose	Access
<code>/</code>	Marketing landing page	public
<code>/login</code> , <code>/register</code> , <code>/forgot-password</code> , <code>/reset-password</code>	Authentication	public
<code>/pair</code>	Device pairing	public (requires code)
<code>/privacy</code> , <code>/terms</code>	Legal pages	public
<code>/onboarding</code>	First-run wizard	admin, pre-onboarding
<code>/admin</code>	Owner dashboard	admin
<code>/pos</code>	Register	admin, cashier, paired device
<code>/inventory</code> , <code>/inventory/receiving</code>	Stock management	admin, cashier (writes gated by permission)
<code>/customers</code>	Customers and utang	admin, cashier
<code>/suppliers</code>	Supplier directory	admin, cashier (no nav entry)
<code>/staff</code>	Staff management	admin + <code>staff.manage</code>
<code>/reports</code>	Reporting incl. Z-Reading	admin + <code>pos.report.view</code>
<code>/settings/*</code>	profile, store, receipts, fees, alerts, backup, devices, plan, audit-log	admin (profile: any staff)
<code>/demo</code>	Read-only demo store	any signed-in
<code>/pricing</code> , <code>/trial-expired</code>	Subscription	admin

28. API architecture

There is no bespoke REST API. Clients use:

1. **PostgREST** over the `public` schema, constrained by RLS.
2. **PostgreSQL functions as RPC** — `checkout_sale`, `void_sale`, `adjust_product_stock`, `set_own_pin`, `generate_pairing_code`, `my_store_billing_state`, `start_trial`, `plan_prices` and others (70 functions in `public`).
3. **Edge Functions** (Deno) for operations requiring elevated rights:

Function	Purpose	Auth
<code>create-cashier</code>	Create staff without exposing admin keys	Admin session
<code>pair-device</code>	Create a device identity from a pairing code	Pairing code
<code>unpair-device</code>	Revoke a device, delete its auth user	Admin + owner PIN
<code>delete-account</code>	Account deletion	Authenticated user
<code>approve-deletion-request</code>	Approve deletion	Platform admin
<code>submit-demo-request</code>	Marketing demo request	Public

Authorization for table access is RLS; for RPCs it is a combination of `SECURITY DEFINER` functions performing their own checks and `EXECUTE` grants. No endpoint is authorized by client-side gating alone.

29. Third-party services

Service	Purpose	Data exchanged	Failure behaviour
Supabase	Database, auth, storage, edge functions	All application data	Application unavailable
Vercel	Web hosting, CDN, TLS	Static assets, requests	Web app unavailable
GitHub Actions	CI and scheduled backup	Source, database dumps	Backups stop; app unaffected
Expo	Mobile build/runtime	Bundles	Mobile builds affected
Google OAuth	Optional sign-in	Identity	Provider not enabled — flow fails
Payment provider	—	—	NOT IMPLEMENTED
Email	Auth emails via Supabase	Addresses, links	Reset/confirm affected
Analytics / monitoring / push	—	—	NOT IMPLEMENTED / [TO BE VERIFIED]

30. Deployment architecture

Development (local) → Staging (Supabase "DellsSoftware-staging")
↓
Production (Supabase "DellsSoftware")

Aspect	Implementation
Branching	<code>feature</code> → <code>dev</code> → <code>main</code> , pull requests required
CI	GitHub Actions: lint, typecheck, build, unit tests
E2E	Playwright specs present (11 spec files)
Migrations	120 SQL migrations applied via Supabase CLI, staging first
Web deployment	Vercel from the repository
Mobile distribution	[TO BE VERIFIED] — no EAS/store configuration identified
Rollback	Git revert + redeploy; database migration rollback is not automated
Release versioning	NOT IMPLEMENTED — see §2

31. Configuration management

Configuration	Location
App mode (Alpha/Production)	VITE_APP_MODE , EXPO_PUBLIC_APP_MODE
Supabase URL / anon key	VITE_SUPABASE_URL , VITE_SUPABASE_ANON_KEY , EXPO_PUBLIC_*
Store configuration	stores (name, address, contact, TIN, permit, VAT status, invoice type, fee config, cashier price-edit flag)
Receipt configuration	Settings → Receipts (with Alpha guardrails that settings cannot override)
Feature flags	feature_flags table
Subscription entitlements	core schema

No credentials, tokens, or keys appear in this document or in the repository. Repository history was audited and is clean of secrets.

32. Logging and monitoring

Area	Status
Application audit log	IMPLEMENTED (§17)
Authentication events	IMPLEMENTED (login/logout audited)
Database / API platform logs	Supabase-provided — retention [TO BE VERIFIED]
Application error monitoring	NOT IMPLEMENTED
Uptime monitoring	[TO BE VERIFIED]
Security alerting	NOT IMPLEMENTED
Log retention policy	NOT DEFINED — FOR BIR VALIDATION

33. Software development and QA

Practice	Status
Version control	Git, monorepo
Branching / PR review	<code>dev</code> → <code>main</code> , PR-based
TypeScript strict checking	IMPLEMENTED — <code>tsc --noEmit</code> clean in both apps
Linting	IMPLEMENTED — <code>oxlint</code> , enforced by pre-commit and pre-push hooks
Unit / integration tests (web)	Vitest — 1,029 tests across 98 files, all passing
Unit / component tests (mobile)	Jest — 275 tests across 36 suites, all passing
End-to-end tests	Playwright — 11 spec files incl. <code>pos-checkout</code> , <code>security</code> , <code>url-bypass</code>
Build verification	CI build on every PR
Coverage measurement	NOT IMPLEMENTED — no coverage gate configured

34. Critical transaction testing

Scenario	Covered	Evidence
Successful sale	YES	Unit + Playwright <code>pos-checkout.spec.ts</code>
Insufficient stock blocks the sale	YES	Server-side validation in <code>checkout_sale()</code>
Credit limit exceeded blocks the sale	YES	<code>CREDIT_LIMIT_EXCEEDED</code>
Successful credit sale updates balance	YES	Atomic within checkout
Failed transaction rolls back	YES	Single-transaction design
Duplicate request produces one sale	YES	Idempotency key; client re-entry guard
Void reverses stock and utang	YES	<code>void_sale()</code> + audit row
Concurrent stock adjustment	YES	<code>adjust_product_stock()</code> , verified under concurrency
Cross-tenant access refused	YES	Live REST testing (§15.3)
Restore from backup	NO	No evidence — §23

35. BIR requirements traceability matrix

Interpretation warning. The rows below map system components to commonly cited BIR expectations for POS/CAS. They are **not** a legal interpretation of RMO 24-2023, RR 7-2024, RR 11-2024, RMC 77-2024, RR 11-2025 or any other issuance. Applicability must be confirmed with the BIR/RDO or a qualified Philippine tax professional.

Requirement area	System component	Current implementation	Evidence	Status	Gap / action
Accreditation / PTU	—	None	—	NOT IMPLEMENTED	Complete BIR process
Official invoice/receipt issuance	Print layer	Order slip marked non-official	§18	NOT IMPLEMENTED (by design)	Validate production layout
Sequential transaction numbering	document_series	Server-allocated, per store, gap-free on failure	§19	PARTIALLY IMPLEMENTED	Confirm format/reset rules
Non-resettable accumulating totals (grand total, Z-counter)	—	Not identified	§21	NOT IMPLEMENTED	Implement + validate
Z-Reading	ZReadingCard	Business date, totals, begin/end receipt no., VAT, voids, reconciliation	§21	PARTIALLY IMPLEMENTED	Add accumulating totals
X-Reading	—	Absent	§21	NOT IMPLEMENTED	Implement
Electronic journal	sales , audit_log	Permanent history; immutable audit	§22	PARTIALLY IMPLEMENTED	Format, sealing, retention
Audit trail of changes	audit_log	12 verified actions, immutable, reprints audited	§17	IMPLEMENTED	Add IP/device; retention
Void / refund controls	void_sale , refunds	Permission-gated, audited, append-only	§10	IMPLEMENTED	Confirm reporting format
VAT computation and presentation	sales VAT columns	Stored and reportable; suppressed on Alpha slips	§18	PARTIALLY IMPLEMENTED	Validate correctness
Data retention	—	No defined period	§23, §32	NOT IMPLEMENTED	Define + enforce
Backup and restore	GH Actions + private bucket	Daily dump; restore untested	§23	PARTIALLY IMPLEMENTED	Document + test restore
Access control / segregation of duties	RLS + RBAC	40/40 RLS, 78 policies, verified matrix	§12, §15	IMPLEMENTED	—
Tamper resistance of records	Immutability trigger	UPDATE/DELETE refused	§17	IMPLEMENTED	Consider hash chaining
Machine identification (MIN/SIN)	—	None	—	NOT IMPLEMENTED	Obtain via BIR process

36. BIR accreditation technical gap analysis

Completed

Tenant isolation and access control; immutable audit trail with reprint logging; atomic transaction processing with duplicate prevention; server-side stock and credit validation; Z-Reading with reconciliation; Alpha print guardrails across every print surface; daily automated backup to private storage; automated test suites and CI.

Critical

Gap	Detail
BIR accreditation/registration	Not started or not evidenced in the repository
Accumulating non-resettable totals	Grand total / Z-counter not implemented
Production invoice format	Not implemented or validated
Suspended-organization sales bypass	§15.5 — open HIGH defect

High

Gap	Detail
X-Reading	Not implemented
Electronic journal format, sealing, retention	Partially implemented
Restore testing	No evidence a backup has ever been restored
Data retention policy	Undefined for transactions, audit, and backups
Google OAuth provider disabled	Advertised sign-in path fails

Medium

Gap	Detail
Audit log lacks IP/device attribution	No columns exist
No error monitoring or security alerting	Not configured
Mobile printing absent	Toggles imply capability that does not exist
Release versioning absent	0.0.0 placeholders
Migration rollback not automated	Manual only

Low

Gap	Detail
Test coverage not measured	No gate
Shift lifecycle incomplete	§11
Mobile offline queue absent	Web only

Missing documentation

Restore/DR runbook · retention policy · production invoice specification · release and versioning policy · privacy notice and DPA records (§16) · signed approver names for this document.

37. System limitations

1. **Alpha.** Not accredited; output is a test document by design.
2. **No X-Reading**, and no accumulating totals.
3. **Mobile cannot print**; its print toggles drive nothing.
4. **No payment gateway** — QR payments are operator-typed references.
5. **No offline mode on mobile.**
6. **No monitoring, alerting, or error reporting.**
7. **Restore has never been tested.**
8. **Google sign-in fails** — provider not enabled.
9. **A suspended organization can still sell** (§15.5).
10. **Version numbers are placeholders.**
11. Several mobile settings persist locally only, mirroring the web application's own unimplemented areas.

38. Future development

Phase	Content
Phase 1 — BIR accreditation preparation	This documentation; Alpha guardrails; gap closure (§36)
Phase 2 — BIR compliance validation	X-Reading, accumulating totals, electronic journal format, retention, production invoice layout, validation with BIR/RDO
Phase 3 — Production release	Accredited invoicing enabled under separate controlled configuration; monitoring; tested DR
Phase 4 — Accounting integration	FUTURE / PLANNED
Phase 5 — Advanced reporting	FUTURE / PLANNED
Phase 6 — AI assistant	FUTURE / PLANNED

Production invoicing must **not** be enabled by an administrator toggle. The mode resolver deliberately refuses to honour a `BIR` configuration value for exactly this reason.

39. User manual summary

Owner: Sign in → Dashboard → Products/Inventory → Customers/Utang → Staff and roles → Reports (incl. Z-Reading) → Settings.

Cashier (own login): Sign in → POS → select products → quantity → discount → customer (credit) → payment → complete sale → print order slip.

Cashier (shared paired device): Device already paired → choose cashier → enter PIN → POS → sell → switch cashier at handover.

40. Screenshot evidence checklist

To be captured for the submission package. Each figure requires a number, title, description, module, and purpose.

Fig.	Screen	Module	Purpose
1	Login	Auth	Access control
2	Registration	Auth	Onboarding
3	Dashboard	Reporting	Overview
4	POS — cart	Sales	Transaction entry
5	POS — payment	Sales	Tender and change
6	Order slip preview	Printing	Alpha markings
7	Printed order slip	Printing	Alpha markings
8	Reprint	Printing	REPRINT + disclaimers
9	Products / Inventory	Inventory	Catalogue and stock
10	Customers / Utang	Credit	Balances
11	Staff and roles	RBAC	Segregation of duties
12	Reports	Reporting	Sales reporting
13	Z-Reading	Reporting	Daily totals
14	Audit log	Governance	Tamper-evident history
15	Settings — receipts	Config	Guardrails not disableable
16	Mobile — home and POS	Mobile	Mobile scope
17	ALPHA TEST MODE indicator	Config	Alpha status visible

41. Technical evidence index

Evidence	Location
Architecture diagram	§5.6
ERD / relationships	§13.2
Database schema	120 migrations under <code>apps/tindakan-pos/supabase/migrations/</code>
Source structure	Monorepo <code>apps/</code>
API documentation	§28
Test results	§33
Sample order slip	§18.1
Audit log evidence	§17
Security configuration	§15, <code>vercel.json</code>
User access matrix	§12
Deployment information	§30
Print guardrail record	<code>ALPHA_PRINT_GUARDRAILS.md</code>
QA reports	<code>apps/tindakan-pos-mobile/ALPHA_MOBILE_*_QA_*.md</code>

No passwords, API secrets, private keys, or credentials are included.

42. Change management

Changes reach production through: feature branch → pull request → review → `dev` → `main` , with CI (lint, typecheck, build, tests) on each PR. Database changes are SQL migrations applied to staging first, then production.

Changes affecting **sales, invoice generation, transaction numbering, tax calculation, reports, audit logs, or database schema** warrant heightened review for accreditation purposes. A formal change-approval record for these areas is **NOT IMPLEMENTED** and is recommended before submission.

43. Data integrity

Protected asset	Mechanism
Sales	Single-transaction checkout; append-only voids; server-assigned numbers
Inventory	Deduction inside the sale transaction; <code>adjust_product_stock()</code> serializes concurrent changes
Payments	Recorded with the sale; refunds append-only
Customer balances	Server-computed only; never client-settable
Transaction numbers	Allocated in-transaction; retained on void; reused on reprint
Audit records	Immutability trigger; UPDATE/DELETE refused
Authorization	RLS on all 40 tables; permissions checked server-side

44. System availability

Aspect	Status
Web hosting	Vercel (managed)
Database	Supabase (managed)
Uptime monitoring	[TO BE VERIFIED]
Failure recovery	Platform-managed; application-level DR not documented
Maintenance procedure	NOT DOCUMENTED
SLA	NOT DEFINED — no availability percentage is claimed

45. Final system inventory

IMPLEMENTED

Authentication · registration · password reset · cashier PIN sessions · device pairing with immediate revocation · RBAC (3 roles, 20 permissions) · RLS on 40/40 tables · products · categories · inventory with receiving, adjustments, counts, warehouses, transfers, purchase orders · POS checkout (atomic, idempotent) · cash/QR/credit payments · voids · refunds · customers and utang with limits, payments and aging · transaction numbering · reports · **Z-Reading** · immutable audit log (12 verified actions incl. reprints) · CSV/JSON export · offline queue (web) · demo store · subscription/trial state · Alpha print guardrails · daily automated backup · CI with 1,304 automated tests.

PARTIALLY IMPLEMENTED

Electronic journal · Z-Reading (no accumulating totals) · shift lifecycle · multi-branch · VAT presentation · backup (untested restore) · Google OAuth.

NOT IMPLEMENTED

X-Reading · accumulating non-resettable totals · production invoice format · payment gateway · mobile printing · mobile offline queue · error monitoring · security alerting · retention policies · MFA on POS · application rate limiting · coverage measurement · release versioning.

FUTURE / PLANNED

BIR-compliant production mode · accounting integration · advanced reporting · AI assistant.

FOR BIR VALIDATION

Accreditation and PTU · invoice/receipt format and wording · numbering format and reset rules · accumulating totals · electronic journal format and sealing · retention periods · VAT computation and presentation · Z-Reading conformity · MIN/SIN and any machine identifiers · QR reference-only tender evidence.

46. Conclusion

Tindahan POS is a working multi-tenant POS with genuine strengths for accreditation purposes: transaction logic and authorization live in the database rather than the client, tenant isolation is verified by live testing, the audit log is immutable and records reprints, and every print surface is guarded so no Alpha document can present itself as an official BIR invoice or receipt.

It is equally clear what is missing. There is no X-Reading, no accumulating non-resettable totals, no validated production invoice format, no tested restore, and no defined retention policy — and the system is not accredited. One open HIGH defect (§15.5) allows a suspended organization to keep transacting.

This document is intended to support the BIR accreditation process by stating accurately what exists and what does not. **It does not assert compliance.**

47. Appendices

A. Verification method. Every quantitative claim was obtained by inspecting the repository at `dev @ 56befe0` or by querying the live staging database. Security claims in §15.3 were produced with real user JWTs over the REST API, because elevated database credentials bypass RLS and would have produced a meaningless result.

B. Related documents. `ALPHA_PRINT_GUARDRAILS.md` — print guardrail implementation record.
`apps/tindahan-pos-mobile/ALPHA_MOBILE_UI_UX_QA_REPORT.md` — UI/UX QA. `apps/tindahan-pos-mobile/ALPHA_MOBILE_QA_TEST_REPORT.md` — RBAC, RLS, paired-device and billing QA. `apps/tindahan-pos/ALPHA_QA_HANDOFF.md` — QA reference.

C. Known documentation discrepancy. `ALPHA_QA_HANDOFF.md` describes 12 permission codes. The database contains **20**, including `settings.store.manage`, which is granted to SUPERVISOR. §12 of this document reflects the database, which is authoritative. The handoff should be corrected.

D. Terminology. "Utang" is Filipino for customer credit and is used throughout the product. "Order slip" is the Alpha document type, deliberately chosen so it is not confused with an official receipt or invoice.

48. Screenshots, user flows and operating procedures

This part demonstrates the system rather than describing it. Every figure is a capture of the **actual running application** at the documented revision. No mock-ups, stock images, generated UI, or screens for functionality that does not exist.

48.1 How the evidence was produced

Web captures	Playwright, <code>apps/tindahan-pos/e2e/capture-screenshots.spec.ts</code> , Chromium at 1440×900, against the built application signed in as a real staff account
Mobile captures	<code>xcrun simctl io ... screenshot</code> , iPhone 15 Pro simulator, Expo development build
Reproducibility	Re-running the capture spec regenerates the whole web set against whatever is currently built
Credentials	Supplied by environment variable at run time; never committed and never visible in a figure

Disclosure — mobile figures. The mobile captures come from an **Expo development build**, so the React Native performance overlay (RAM / JS frame rate) is visible in the top-left. That overlay is a development artifact and is **not part of the shipped interface**. It is left in rather than edited out, because editing screenshots would breach the requirement that evidence be unaltered captures of the real application.

48.2 Screenshot evidence table

Fig.	App	Screen / flow	Role	Purpose	Status
1	Web	Login	Public	Authentication entry point	CAPTURED
2	Web	Landing page	Public	Product presentation	CAPTURED
3	Web	POS / register	Admin, Cashier	Sales entry	CAPTURED
4	Web	POS with cart	Cashier	Cart, quantity, totals	CAPTURED
5	Web	Dashboard	Admin	Business overview, ALPHA indicator	CAPTURED
6	Web	Inventory	Admin, Cashier	Stock levels	CAPTURED
7	Web	Customers / utang	Admin, Cashier	Credit balances	CAPTURED
8	Web	Reports	Admin + pos.report.view	Sales reporting, Z-Reading	CAPTURED
9	Web	Settings → Receipts	Admin	Alpha order slip and guardrails	CAPTURED
10	Web	Audit log	Admin	Tamper-evident history	CAPTURED
11	Web	Staff and roles	Admin + staff.manage	Segregation of duties	CAPTURED
12	Web	Settings → Store	Admin	Store and BIR fields	CAPTURED
13	Mobile	Settings → Receipts	Admin	Alpha order slip and guardrails	CAPTURED
—	Mobile	Home, POS, settings menu	Admin	Mobile navigation and register	PENDING CAPTURE
—	Web	Completed-sale receipt modal, reprint	Cashier	Transaction document and reprint marker	PENDING CAPTURE — requires completing a live sale against staging
—	Web	Registration, onboarding, shift	Admin	Signup and shift lifecycle	PENDING CAPTURE

Figures are stored in docs/screenshots/. Pending items are listed rather than omitted, per the requirement to identify missing evidence.

48.3 Figure index

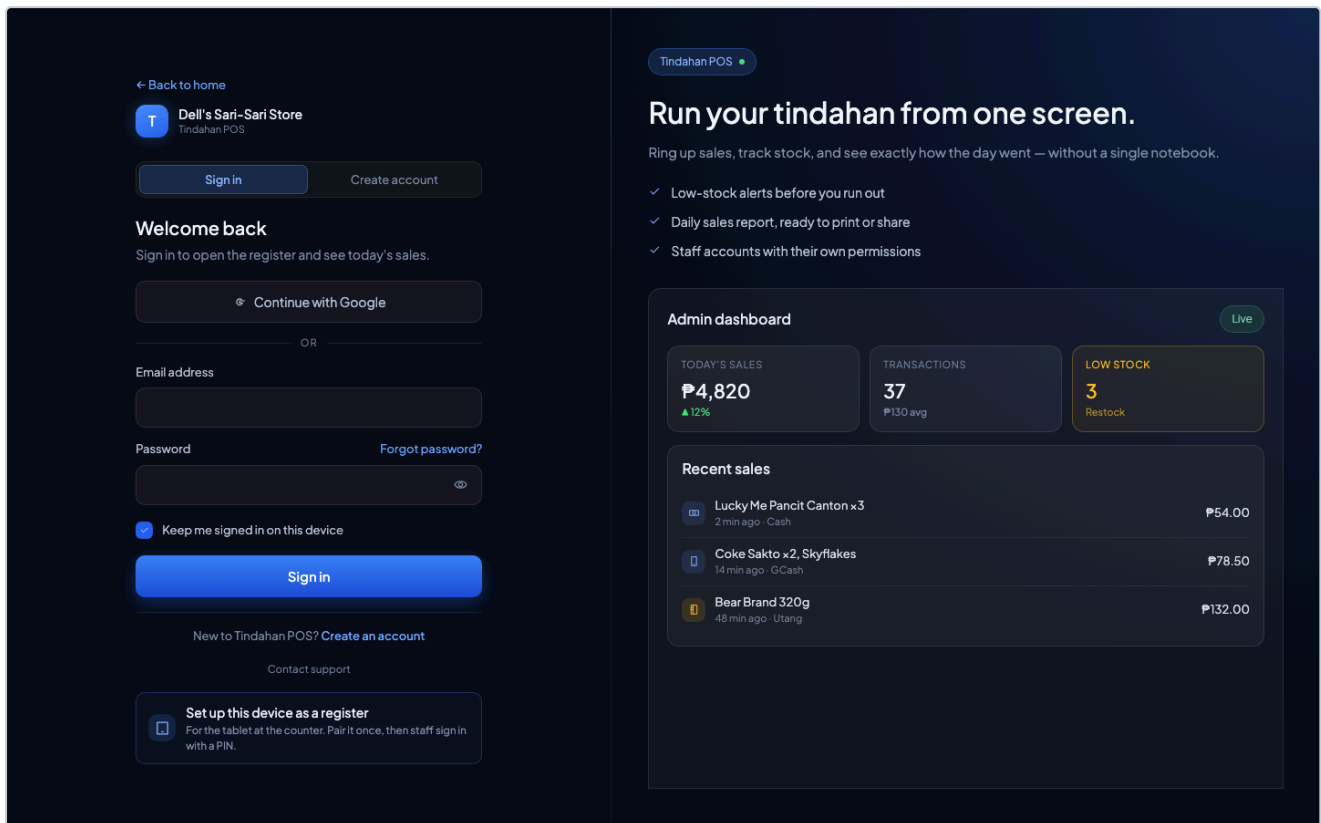


Figure 1 — Login.

Figure 1 — Login. Module: Authentication. Role: all users. Email and password fields, "Forgot password", Google sign-in (provider not enabled, §29), and "Set it up as a register" for device pairing.

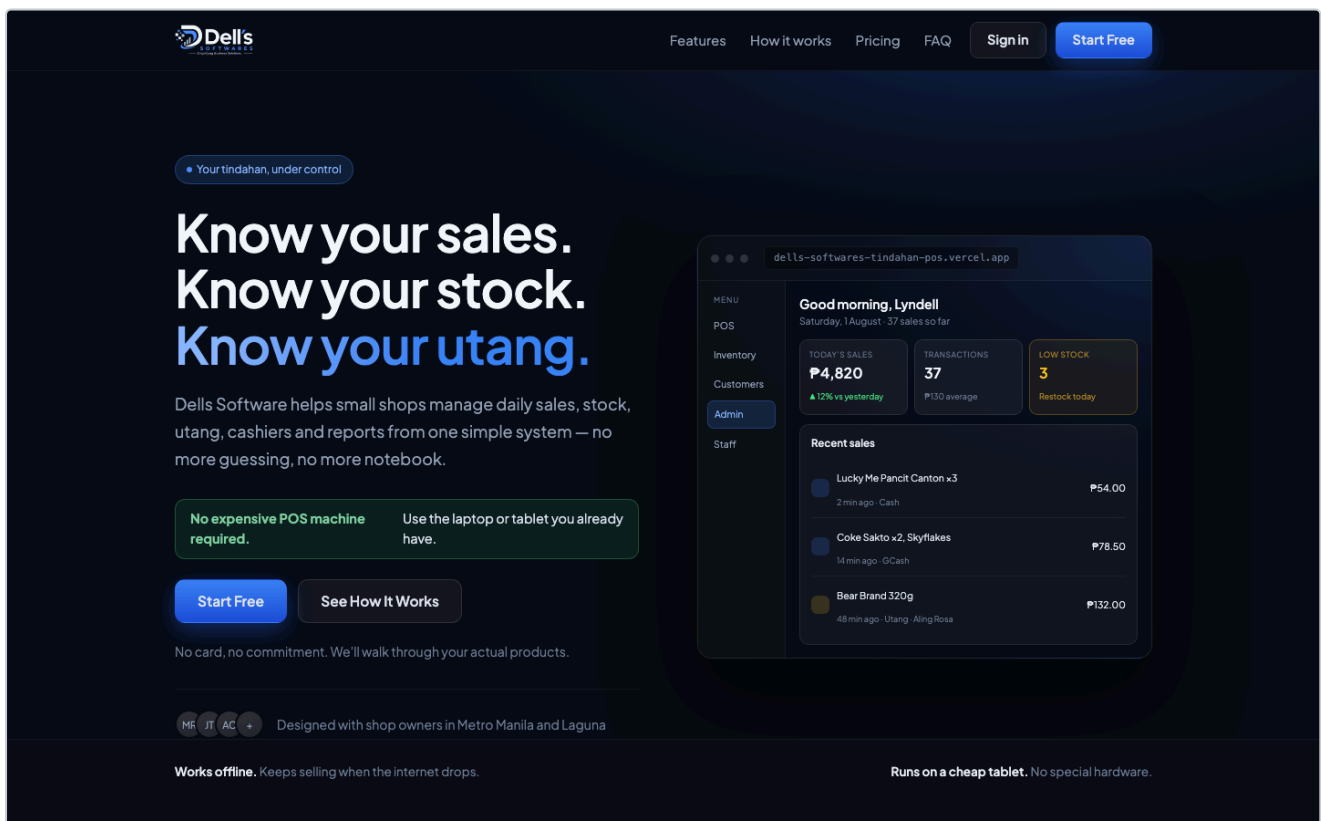


Figure 2 — Landing page.

Figure 2 — Landing page. Public marketing page served at / .

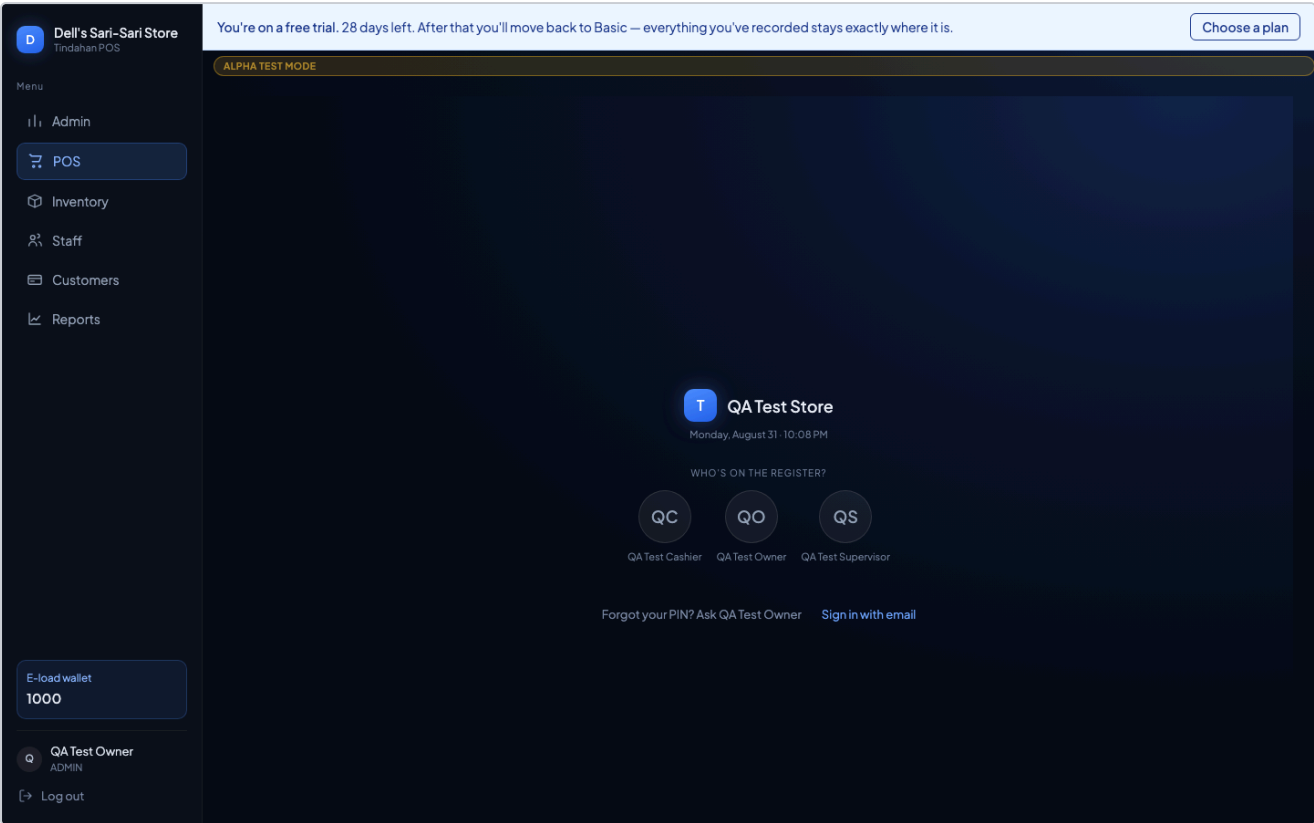


Figure 3 — POS / register.

Figure 3 — POS / register. Module: Sales. The register with product tiles, search, and category filters.

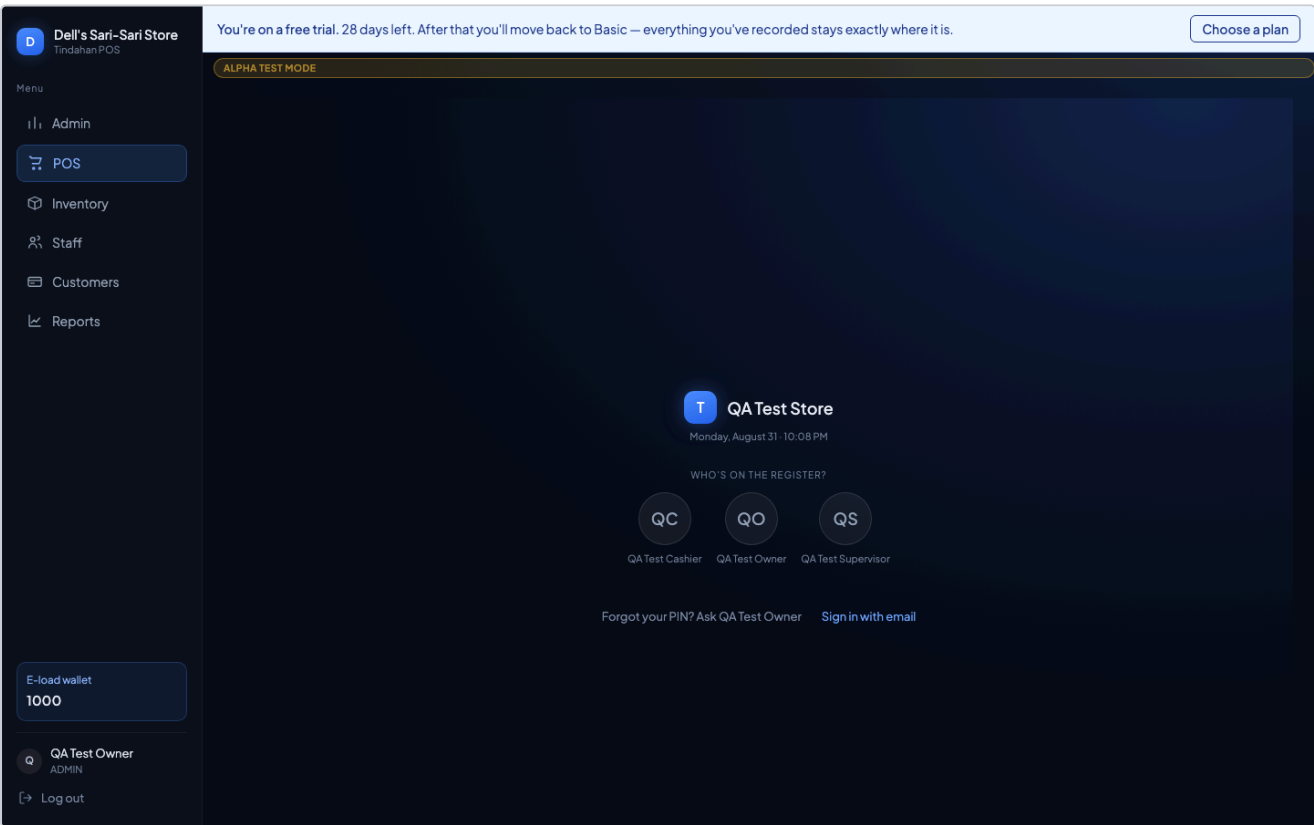


Figure 4 — POS with cart.

Figure 4 — POS with cart. Cart lines, quantities and running total before payment.

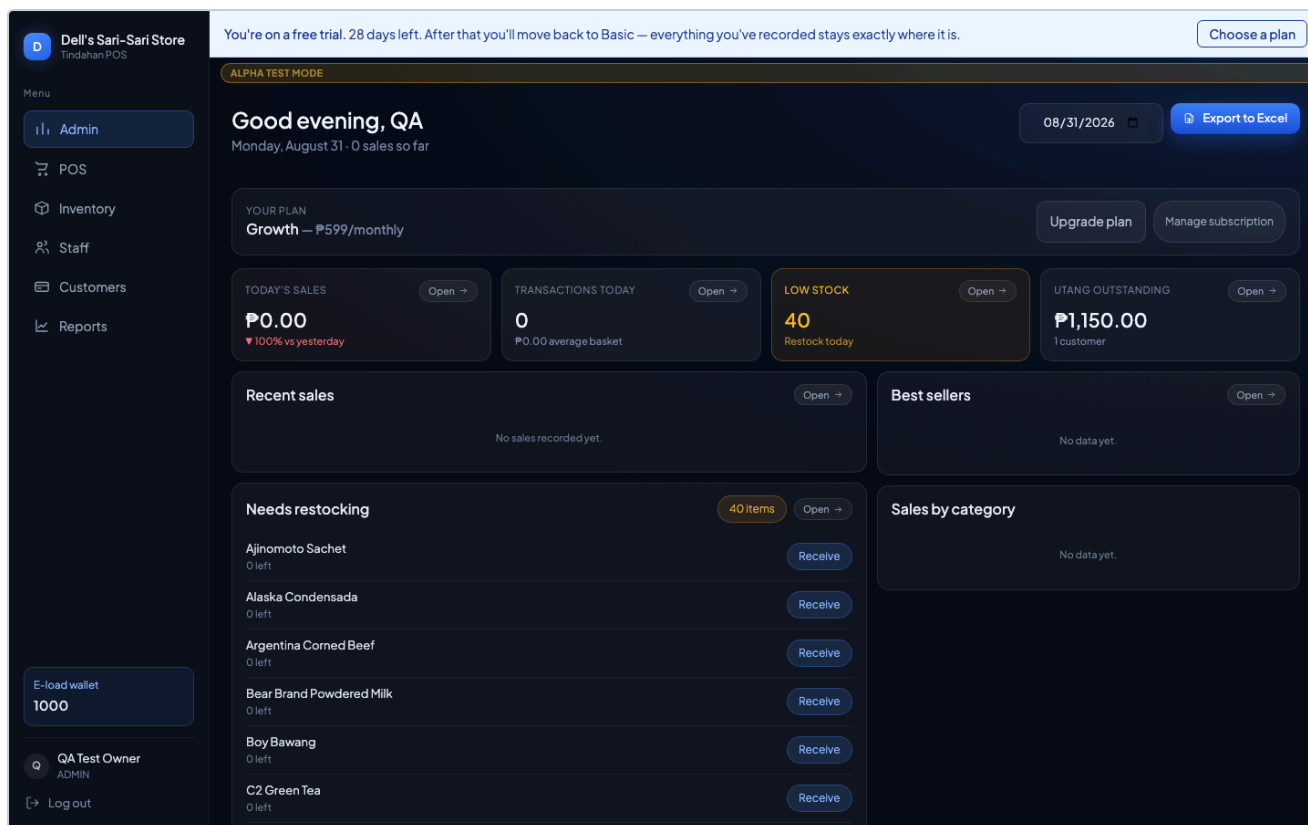


Figure 5 — Dashboard.

Figure 5 — Dashboard. Module: Reporting. Today's sales, transaction count, low stock, utang outstanding, restocking list. The **ALPHA TEST MODE** indicator (§13 of the guardrail specification) is visible in the top bar, as is the trial banner.

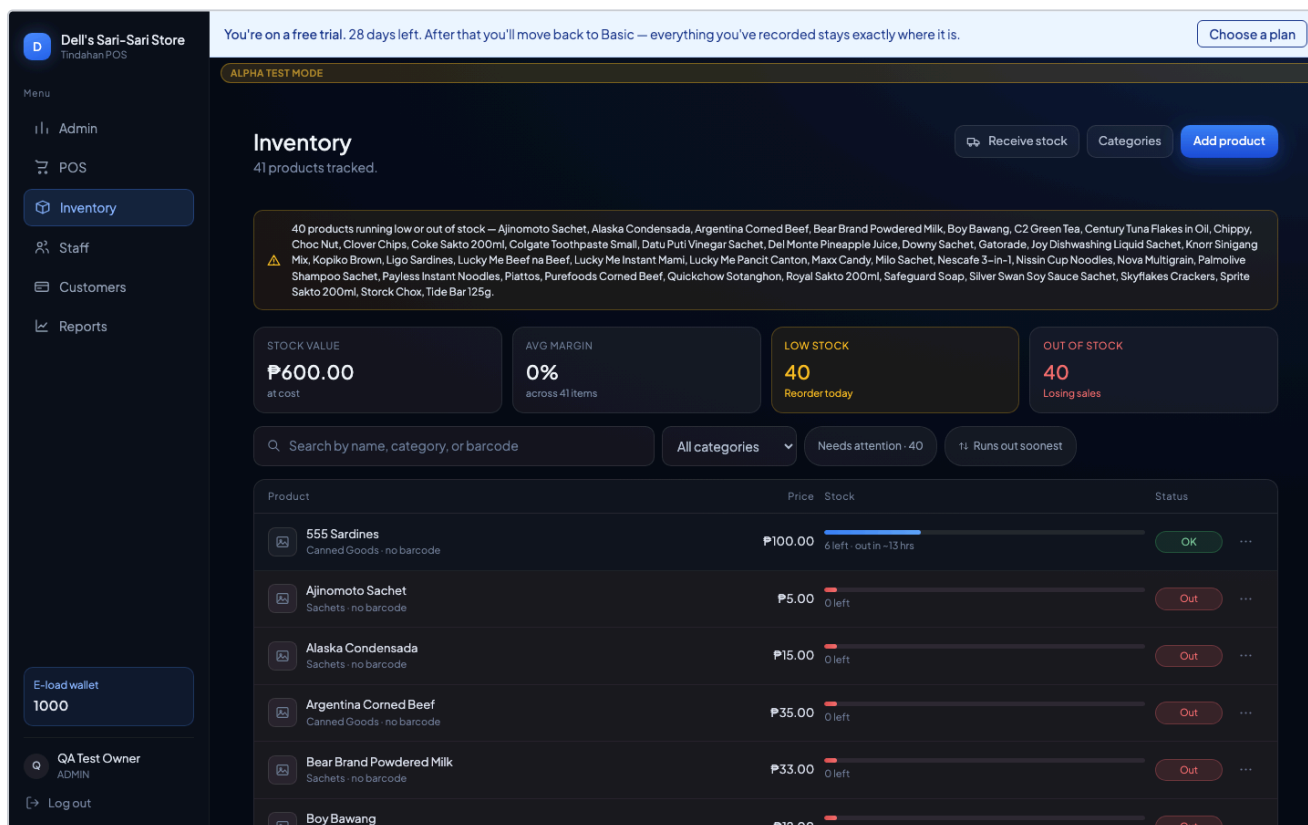


Figure 6 — Inventory.

Figure 6 — Inventory. Product list with stock levels and restocking.

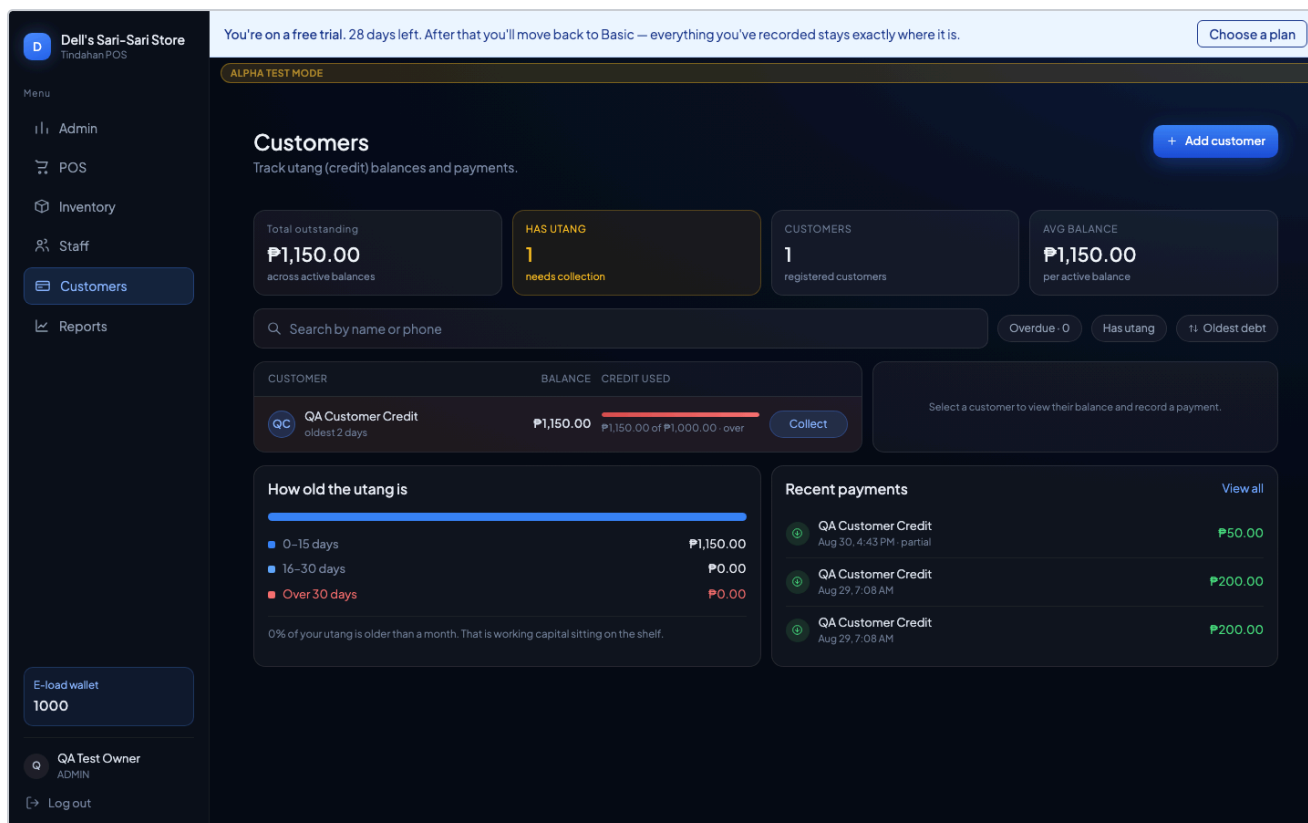


Figure 7 — Customers / utang.

Figure 7 — Customers / utang. Customer balances and credit.

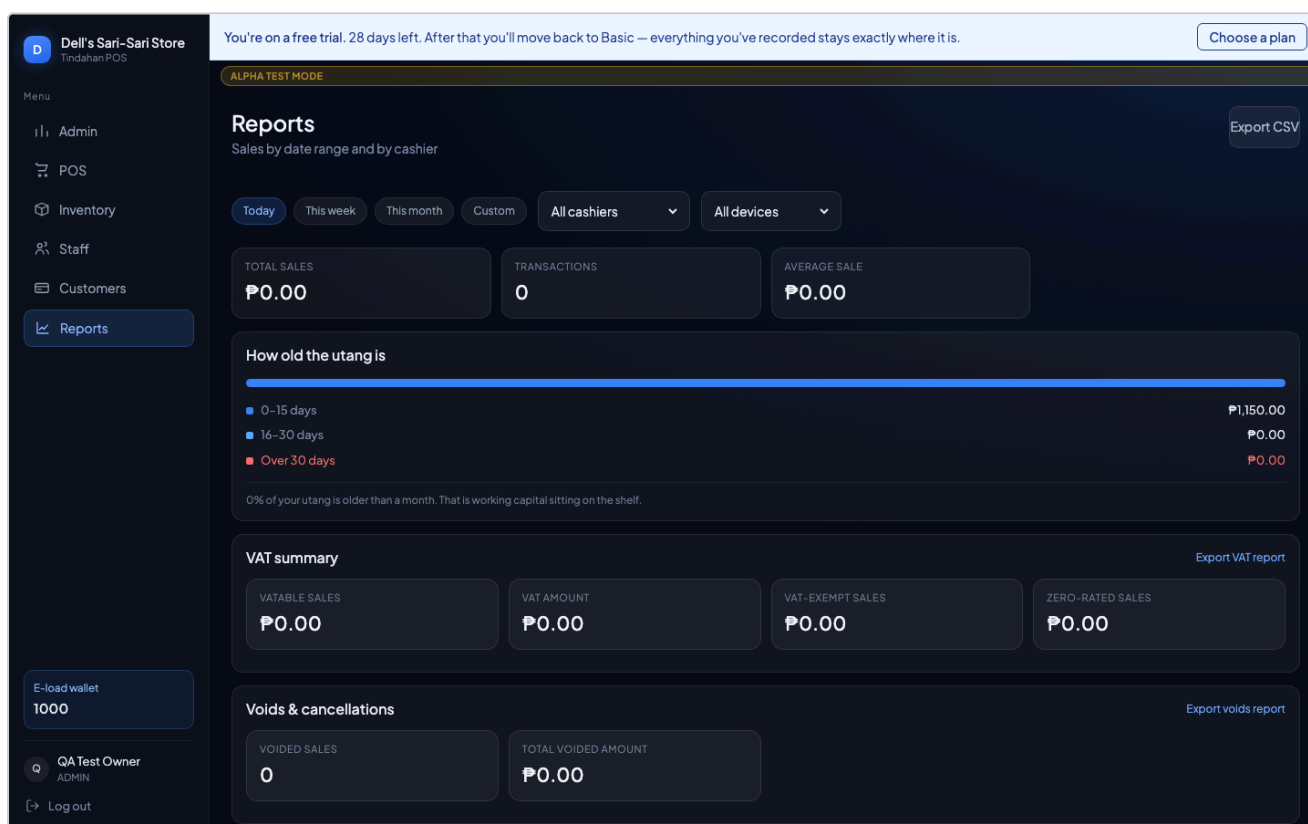


Figure 8 — Reports.

Figure 8 — Reports. Date/cashier/device filters, sales tables, VAT summary, void and refund summaries, and the Z-Reading card.

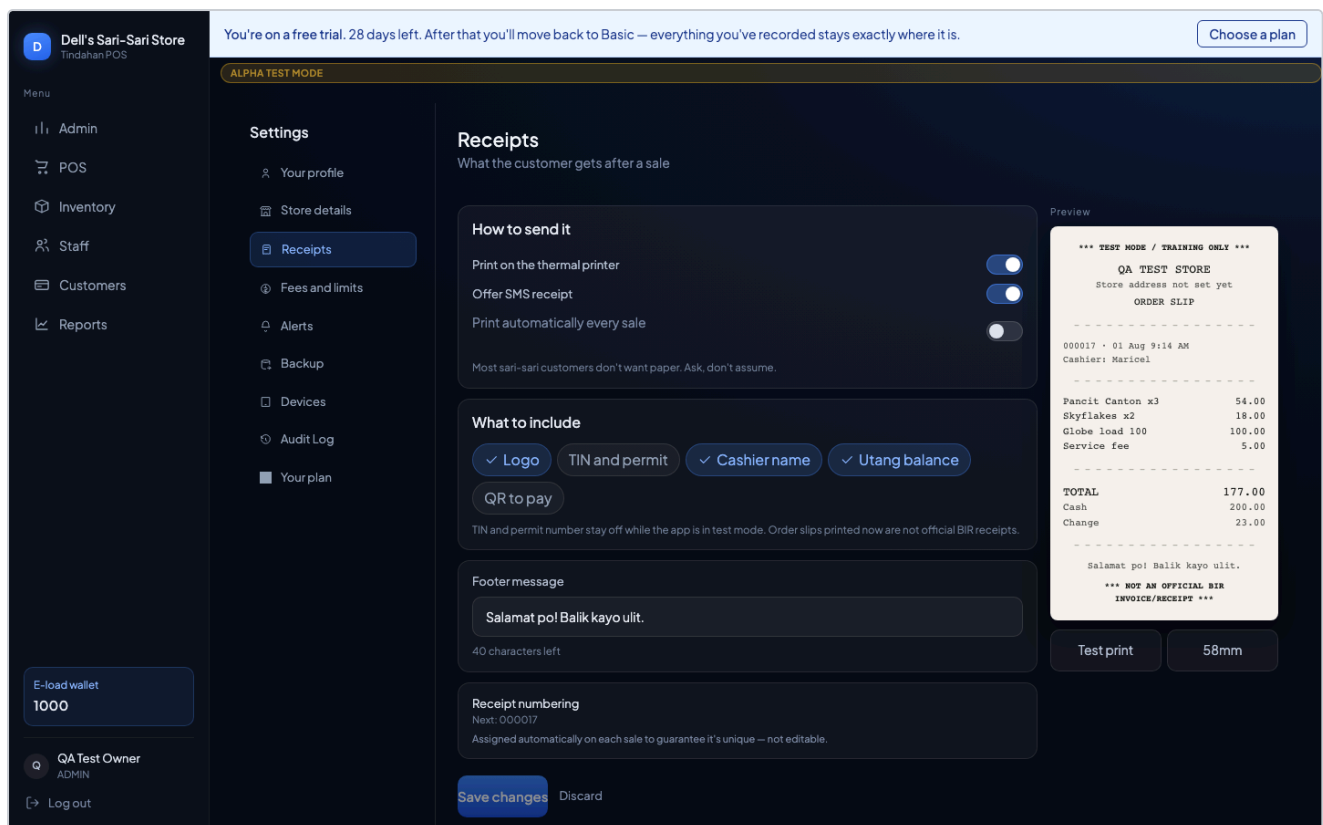


Figure 9 — Settings → Receipts (web).

Figure 9 — Settings → Receipts (web). The most BIR-relevant web figure. Shows the order-slip preview carrying *** TEST MODE / TRAINING ONLY ***, the document type **ORDER SLIP**, and *** NOT AN OFFICIAL BIR INVOICE/RECEIPT *** below the operator's own footer message. The **TIN and permit chip is off and locked**, with the explanation that registration identifiers stay off in test mode. Receipt numbering shows the next server-assigned number.

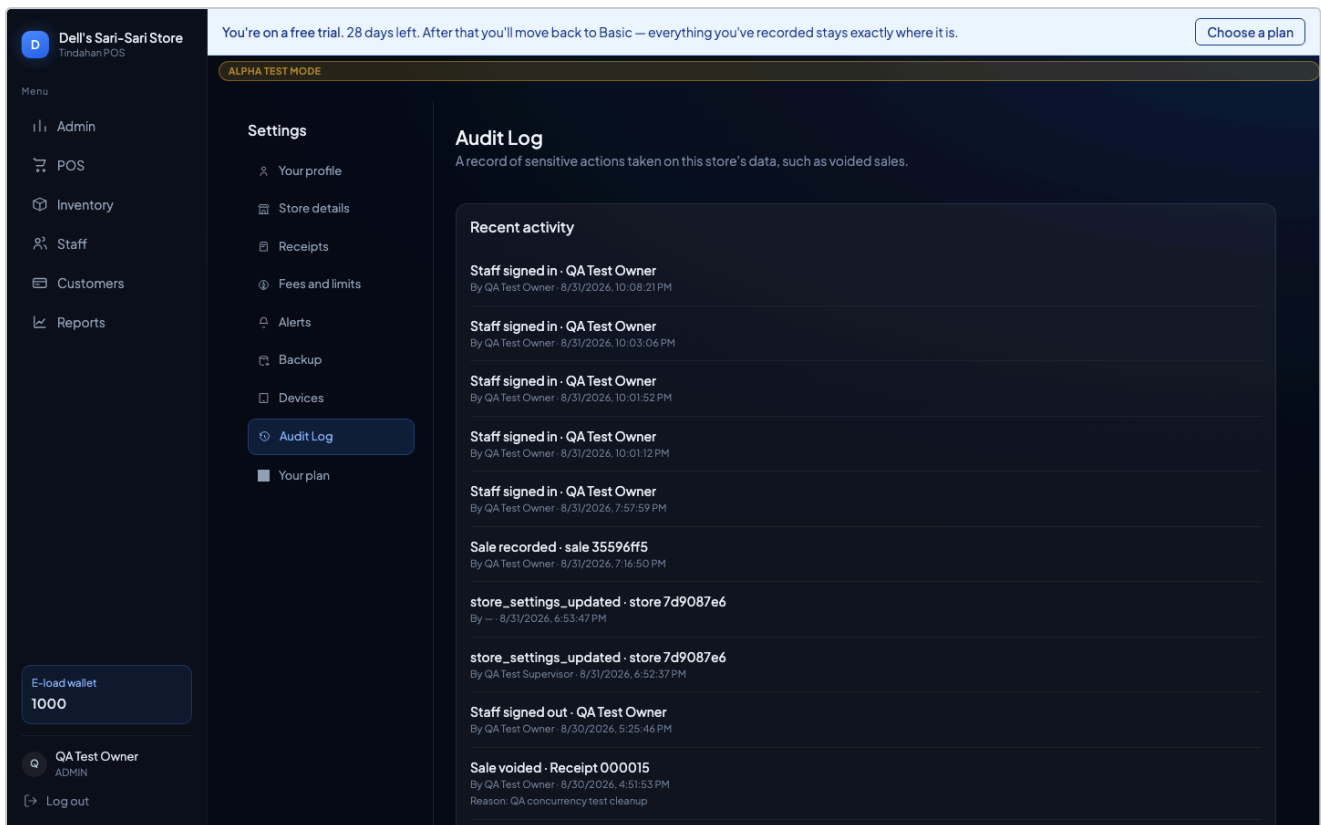


Figure 10 — Audit log.

Figure 10 — Audit log. Module: Governance. Recorded actions with actor and timestamp (§17).

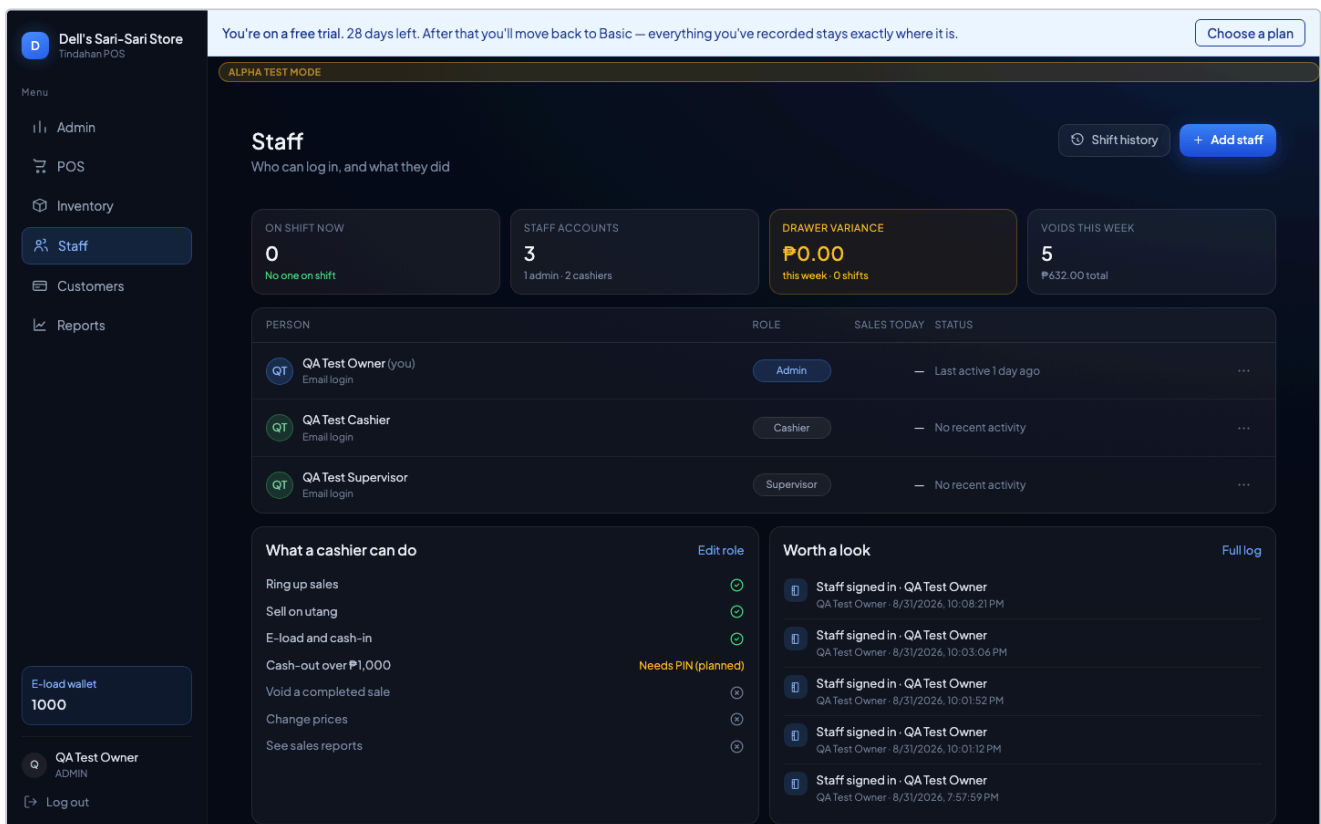


Figure 11 — Staff and roles.

Figure 11 — Staff and roles. Staff accounts and role assignment (§12).

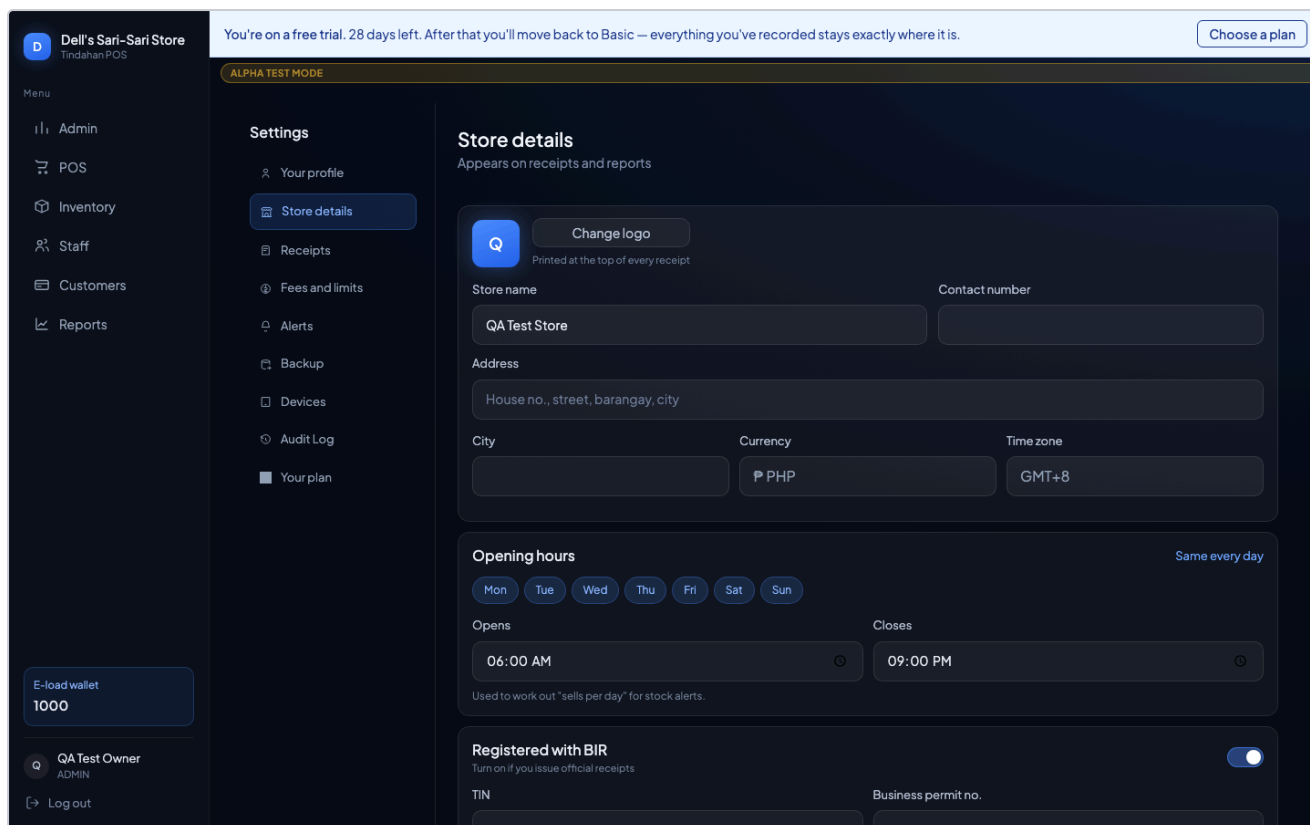


Figure 12 — Settings → Store.

Figure 12 — Settings → Store. Store identity and BIR-related fields (TIN, permit number, BIR-registered flag, VAT status).

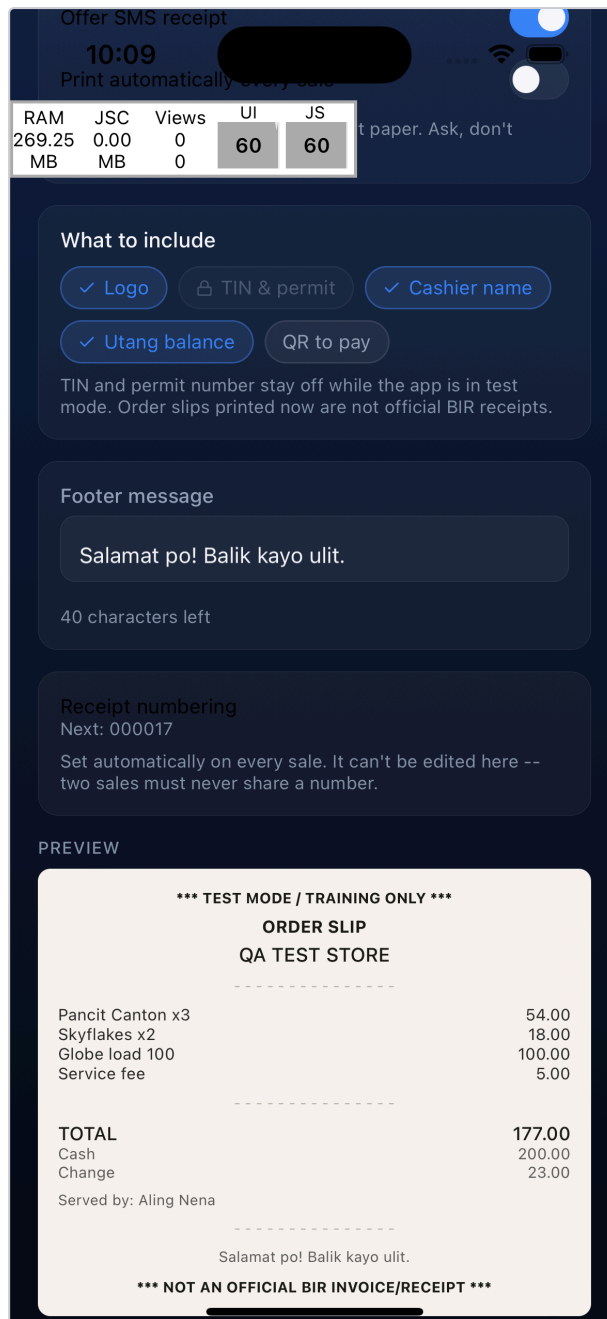


Figure 13 — Settings → Receipts (mobile).

Figure 13 — Settings → Receipts (mobile). The mobile equivalent of Figure 9, showing the same guardrails on the mobile preview and the same locked TIN chip. Development-build overlay present, see §48.1.

48.4 Flow — user login

```
Open application
↓
/login — email + password
↓
Supabase Auth (GoTrue) issues JWT
↓
auth_store_id() resolves the tenant
↓
staff row → role; staff_roles → permissions
↓
admin → /admin (dashboard)    cashier → /pos (register)
↓
audit_log 'staff_logged_in'
```

Verified behaviour: failed sign-in returns an explicit error and creates no session. An onboarded admin lands on `/admin`; a cashier lands on `/pos`. Route access is gated client-side **and** independently by RLS (§15.2).

Evidence: Figure 1, Figure 3, Figure 5.

48.5 Flow — POS sales transaction

The authoritative sequence is in §8.4. In operating terms:

```
Cashier at the register (Fig 3)
↓ select products / scan barcode
Cart with quantities and total (Fig 4)
↓ optional discount; customer for a credit sale
Choose payment: cash · QR (reference no.) · credit
↓
checkout_sale() — one database transaction
↓
sale + items written · stock deducted · customer balance updated
receipt number allocated · audit_log 'sale_created'
↓
Order slip rendered (Fig 9) — marked TEST MODE / ORDER SLIP
↓
Dashboard, reports and audit log reflect the sale (Figs 5, 8, 10)
```

Insufficient stock. Validation happens inside `checkout_sale()`, not in the client. A sale that exceeds available stock is rejected, no sale row is created, and no stock is deducted — there is no partial state, because the whole operation is one transaction.

Insufficient credit. A credit sale beyond the customer's limit raises `CREDIT_LIMIT_EXCEEDED` and is rejected. A null limit means unlimited. An owner PIN override exists, with lockout.

Duplicate submission. The client disables the button while a checkout is in flight and re-checks on entry; the server additionally de-duplicates on `p_client_request_id`. A double-tap cannot produce two sales.

48.6 Flow — customer credit and collection

```
Credit sale → checkout_sale() → customers.balance increases
Collection → credit_payments row (records resulting balance)
              → balance decreases → aging and history update
```

Balances are server-computed and never client-settable (§9). Evidence: Figure 7.

48.7 Flow — reprint

```
Reports / sales history → select sale → reprint
↓
Same Receipt component, isReprint = true
↓
*** TEST MODE / TRAINING ONLY ***      (header, unchanged)
*** REPRINT ***                        (added)
... document ...
*** NOT AN OFFICIAL BIR INVOICE/RECEIPT *** (footer, unchanged)
↓
audit_log 'receipt_reprinted'
```

A reprint **cannot** bypass the disclaimers: the header renders before the reprint marker and the footer after everything else, from the same template. Reprints are audited — `receipt_reprinted` is one of the 12 verified audit actions (§17). Evidence for the marker: §18.2 and the Receipt test suite; screenshot **PENDING CAPTURE**.

48.8 Role-based operating procedures

Owner / Administrator (`staff.role = 'admin'` , all 20 permissions)

```
Sign in → Dashboard (Fig 5) → Inventory (Fig 6) → Customers (Fig 7)
      → Staff and roles (Fig 11) → Reports incl. Z-Reading (Fig 8)
      → Settings: store, receipts, fees, alerts, backup, devices,
        plan, audit log (Figs 9, 10, 12)
```

Supervisor (`staff.role = 'cashier'` + SUPERVISOR, 15 permissions)

```
Sign in → POS → sell · void · refund
      → Inventory: manage products, adjust, receive, count
      → Customers → Reports
      (no staff management, no organization/branch settings)
```

Cashier (`staff.role = 'cashier'` , no granular permissions)

```
Sign in (or PIN on a shared device) → POS → sell
      → cash · QR · credit within the customer's limit
      → print order slip
      (no reports, no staff, no settings – verified server-side:
       a cashier's API read of `sales` returns zero rows, §15.2)
```

Paired device (no `staff` row)

```
Pair once with a code → choose cashier → PIN → register only
      (verified: no sales history, no staff, no audit log, §11)
```

There is no separate "Manager" or "Staff" role. The specification's example matrix lists them; this system implements Owner/Admin, Supervisor, Cashier and Paired Device only. They are not documented here because they do not exist.

48.9 End-to-end scenario — cash sale

#	Step	System behaviour	Evidence
1	Cashier signs in	JWT issued; <code>staff_logged_in</code> audited	Fig 1
2	Opens the register	Catalogue loaded, store-scoped by RLS	Fig 3
3	Adds items	Cart totals computed client-side for display	Fig 4
4	Takes ₱200 cash for a ₱177 sale	Change ₱23 computed	Fig 9 (slip)
5	Completes the sale	One transaction: sale + items + stock + number + audit	§8.4
6	Document produced	Order slip, both disclaimers, ORDER SLIP type	Fig 9
7	Reporting updates	Dashboard and reports reflect the sale	Figs 5, 8
8	Audit written	<code>sale_created</code>	Fig 10

48.10 BIR-relevant lifecycle demonstration

```
Product (Fig 6)
↓
POS (Figs 3, 4)
↓
Sale — checkout_sale(), atomic (§8)
↓
Payment recorded · inventory deducted · customer balance updated
↓
Transaction document — ORDER SLIP, test-marked (Figs 9, 13)
↓
Reports incl. Z-Reading (Fig 8)
↓
Audit trail incl. reprints (Fig 10)
```

ALPHA / TEST OUTPUT — what exists today. An order slip marked `* TEST MODE / TRAINING ONLY *` and `* NOT AN OFFICIAL BIR INVOICE/RECEIPT *`, with no VAT breakdown and no registration identifiers.

FUTURE BIR PRODUCTION OUTPUT — FOR BIR VALIDATION. An accredited invoice or receipt, its required content and layout, accumulating non-resettable totals, X-Reading, and any electronic transmission. None of that is implemented, and none of it is claimed.

48.11 Completeness checklist

Item	Status
Login flow documented	DONE (§48.4)
POS flow documented	DONE (§48.5)
Payment flows documented	DONE (§48.5, §10)
Credit/utang flow documented	DONE (§48.6)
Customer payment flow documented	DONE (§48.6)
Reprint flow documented	DONE (§48.7)
Reports documented	DONE (§20, §21)
Role-based procedures documented	DONE (§48.8)
End-to-end transaction demonstrated	DONE (§48.9)
BIR lifecycle demonstrated	DONE (§48.10)
Print output documented	DONE (§18, Figs 9, 13)
Audit trail documented	DONE (§17, Fig 10)
Major web screens captured	DONE — 12 figures
Major mobile screens captured	PARTIAL — 1 of 4
Registration / onboarding captured	NOT DONE
Shift lifecycle captured	NOT DONE — feature itself is partial (§11)
Completed-sale receipt and reprint captured	NOT DONE
Screenshots match the documented revision	YES
No fabricated screenshots	YES
Missing features identified	YES (§36, §37)
BIR gaps identified	YES (§35, §36)

48.12 Product setup, inventory receiving and shift — status

Product setup and inventory receiving are implemented (§7.3) and visible in Figure 6. Step-by-step figures for adding a product and receiving stock are **PENDING CAPTURE**.

Cashier shift is **PARTIALLY IMPLEMENTED** (§11). Cashier PIN sessions exist and are audited (`cashier_session_started` / `_ended`), but a formal shift lifecycle with declared opening float, blind close and variance sign-off was not verified. The specification's shift flow is therefore **not** documented as implemented — documenting it would assert functionality that has not been confirmed.